



Pyspark

Supercharge

your

Interview

Preparation



Top Spark Interview Questions:

Q1) What is Apache Spark?

Apache Spark is an **Analytics engine for processing data at large-scale**. It provides high-level APIs (Application Programming Interface) in multiple programming languages like Java, Scala, Python and R. It provides an optimized engine that supports general execution of graphs. It also supports an upscale set of higher-level tools including Spark SQL for SQL and structured processing of data, MLlib for machine learning, GraphX for graph processing, and Structured Streaming for incremental computation and stream processing.

Q2) What is an RDD in Apache Spark?

RDD Stands for **Resilient Distributed Dataset**. From a top-level perspective, every Spark application consists of a driver program that runs the user's main function and executes various parallel operations on a cluster. **RDD is an abstract term provided by Spark, which means a collection of elements partitioned across the nodes of the cluster that can be operated on in parallel so they automatically recover from node failures making them fault-tolerant.**

RDD's can be created in two ways:

1. **Parallelizing** an **existing collection** in your driver program.
2. **Referencing** a dataset from an **external storage system**, such as a shared filesystem, HDFS, HBase, or any data source offering a Hadoop Input Format.

RDD's support two types of operations:

1. **Transformations**: which create a new dataset from an existing one, e.g.: MAP.
2. **Actions**: which return a value to the driver program after running a computation on the dataset. e.g.: REDUCE.

All transformations in Spark are lazy, meaning, they do not compute their results right away. Instead, they just remember the transformations applied to some base dataset. The transformations are only computed when an action requires a result to be returned to the driver program. This design enables Spark to run more efficiently.

One of the most important capabilities in Spark is **persisting** (or **caching**) a dataset in memory across operations. When you persist an RDD, each node stores any partitions of it that it computes in memory and reuses them in other actions on that dataset (or

datasets derived from it). This allows future actions to be much faster (often by more than 10x).

Q3) Why use Spark on top of Hadoop?

While Apache Hadoop is a framework which allows us to **store and process big data** in a distributed environment, Apache Spark is only a **data processing engine** developed to provide **faster and easy-to-use analytics** than Hadoop MapReduce. So, we store data in the Hadoop File System and use YARN for resource allocation on top of which we use Spark for processing data fast. Hadoop Map Reduce can't process data fast and Spark doesn't have its own Data Storage so they both compensate for each other's drawbacks and come strong together.

Note: We can use Spark Core or Hadoop Map Reduce as a Computing Engine.

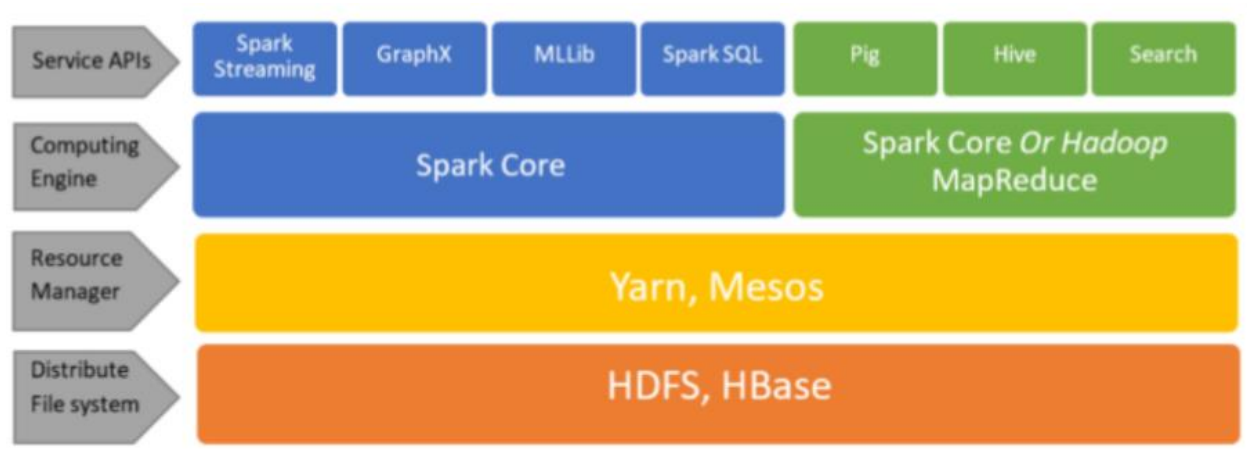


Image reference: Towards Data Science: Jeroen Schmidt.

Q4) How to install Spark on windows?

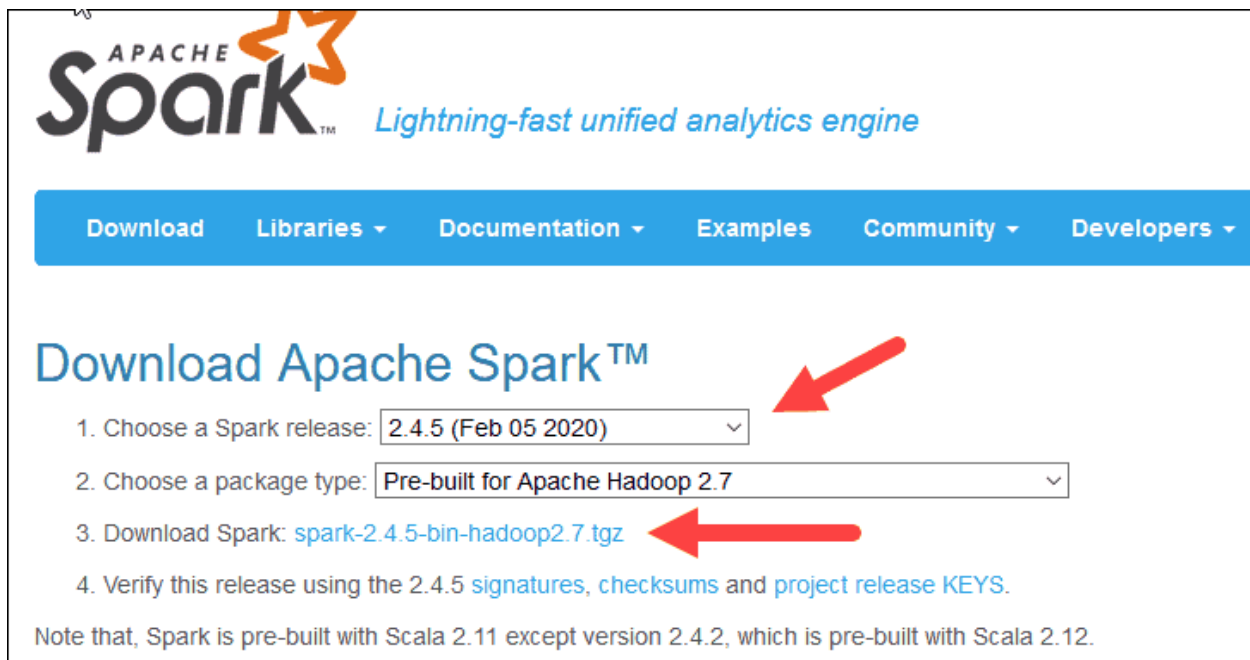
Prerequisites:

1. A system running Windows 10
2. A user account with administrator privileges (required to install software, modify file permissions, and modify system PATH)
3. Command Prompt or Powershell
4. A tool to extract .tar files, such as 7-Zip
5. Already installed Java
6. Already installed Python

Install Apache Spark on Windows

Step 1: Download Apache Spark

1. Open a browser and navigate to <https://spark.apache.org/downloads.html>.
2. Under the *Download Apache Spark* heading, there are two drop-down menus. Use the current non-preview version.
 - In our case, in **Choose a Spark release** drop-down menu select **2.4.5 (Feb 05 2020)**.
 - In the second drop-down **Choose a package type**, leave the selection **Pre-built for Apache Hadoop 2.7**.
3. Click the **spark-2.4.5-bin-hadoop2.7.tgz** link.



4. A page with a list of mirrors loads where you can see different servers to download from. Pick any from the list and save the file to your Downloads folder.

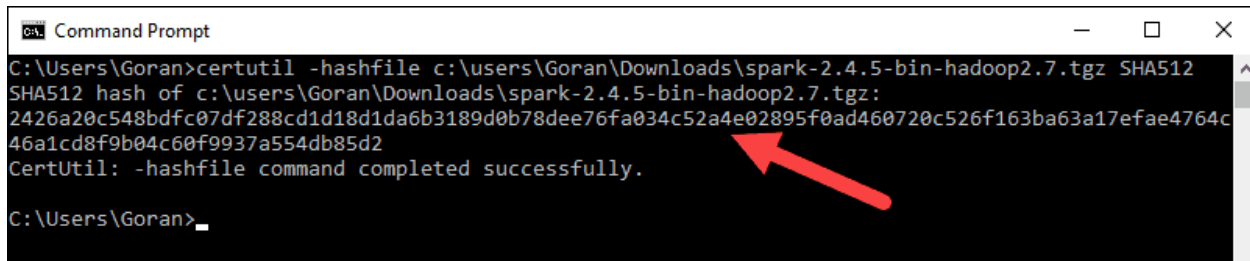
Step 2: Verify Spark Software File

1. Verify the integrity of your download by checking the **checksum** of the file. This ensures you are working with unaltered, uncorrupted software.
2. Navigate back to the *Spark Download* page and open the **Checksum** link, preferably in a new tab.

3. Next, open a command line and enter the following command:

```
certutil -hashfile c:\users\username\Downloads\spark-2.4.5-bin-hadoop2.7.tgz SHA512
```

4. Change the username to your username. The system displays a long alphanumeric code, along with the message **Certutil: -hashfile completed successfully.**



```
Command Prompt
C:\Users\Goran>certutil -hashfile c:\users\Goran\Downloads\spark-2.4.5-bin-hadoop2.7.tgz SHA512
SHA512 hash of c:\users\Goran\Downloads\spark-2.4.5-bin-hadoop2.7.tgz:
2426a20c548bdfc07df288cd1d18d1da6b3189d0b78dee76fa034c52a4e02895f0ad460720c526f163ba63a17efae4764c
46a1cd8f9b04c60f9937a554db85d2
CertUtil: -hashfile command completed successfully.

C:\Users\Goran>_
```

5. Compare the code to the one you opened in a new browser tab. If they match, your download file is uncorrupted.

Step 3: Install Apache Spark

Installing Apache Spark involves **extracting the downloaded file** to the desired location.

1. Create a new folder named *Spark* in the root of your C: drive. From a command line, enter the following:

```
cd \
```

```
mkdir Spark
```

2. In Explorer, locate the Spark file you downloaded.

3. Right-click the file and extract it to *C:\Spark* using the tool you have on your system.

4. Now, your *C:\Spark* folder has a new folder *spark-2.4.5-bin-hadoop2.7* with the necessary files inside.

Step 4: Add winutils.exe File

Download the **winutils.exe** file for the underlying Hadoop version for the Spark installation you downloaded.

1. Navigate to this URL <https://github.com/cdarlint/winutils> and inside the **bin** folder, locate **winutils.exe**, and click it.



 mapred	some binaries from 273 to 311
 mapred.cmd	some binaries from 273 to 311
 rcc	some binaries from 273 to 311
 winutils.exe	fixed exe and lib 265-312
 winutils.pdb	fixed exe and lib 265-312
 yarn	some binaries from 273 to 311
 yarn.cmd	some binaries from 273 to 311

2. Find the **Download** button on the right side to download the file.

3. Now, create new folders **Hadoop** and **bin** on C: using Windows Explorer or the Command Prompt.

4. Copy the winutils.exe file from the Downloads folder to **C:\hadoop\bin**.

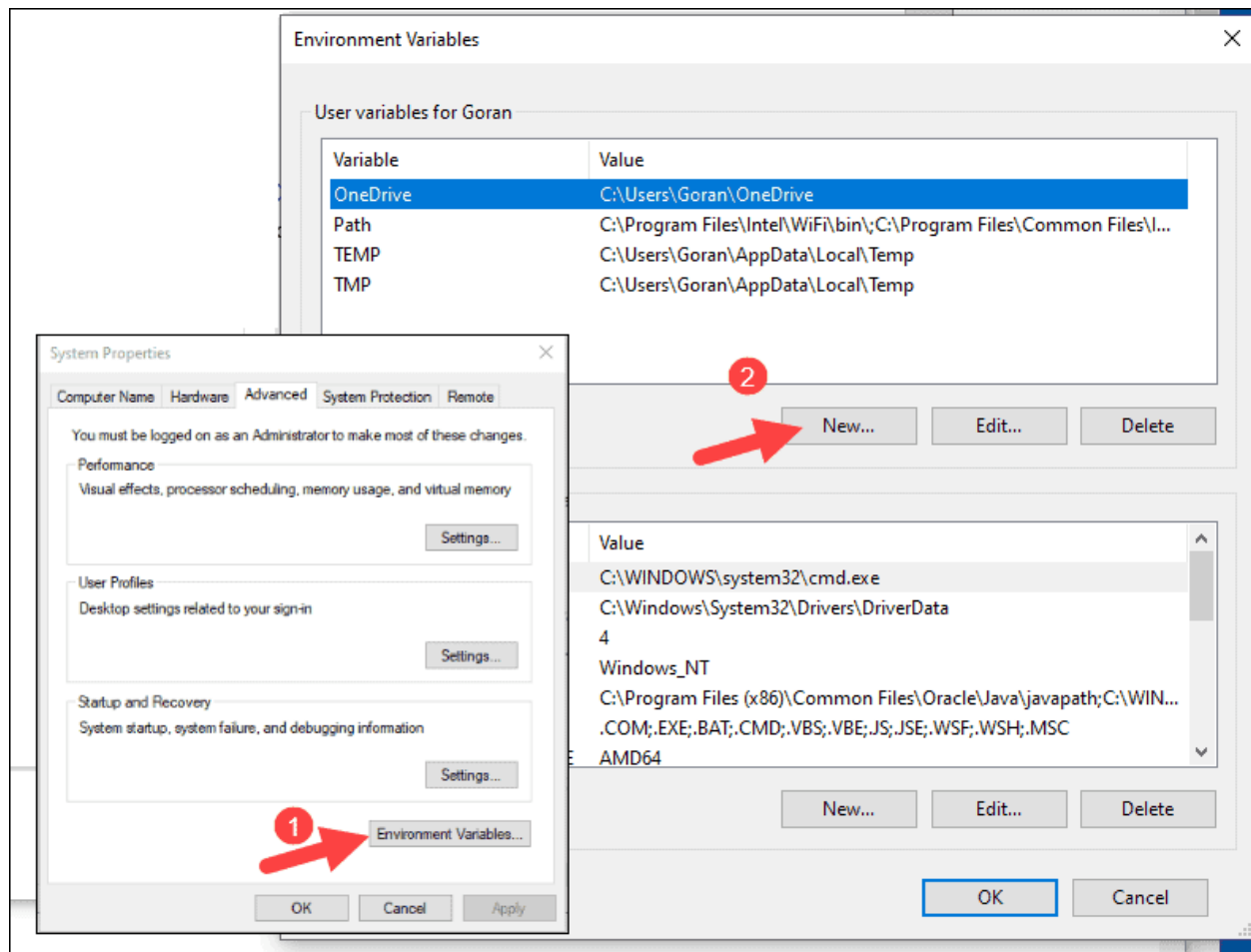
Step 5: Configure Environment Variables

This step adds the Spark and Hadoop locations to your system PATH. It allows you to run the Spark shell directly from a command prompt window.

1. Click **Start** and type *environment*.

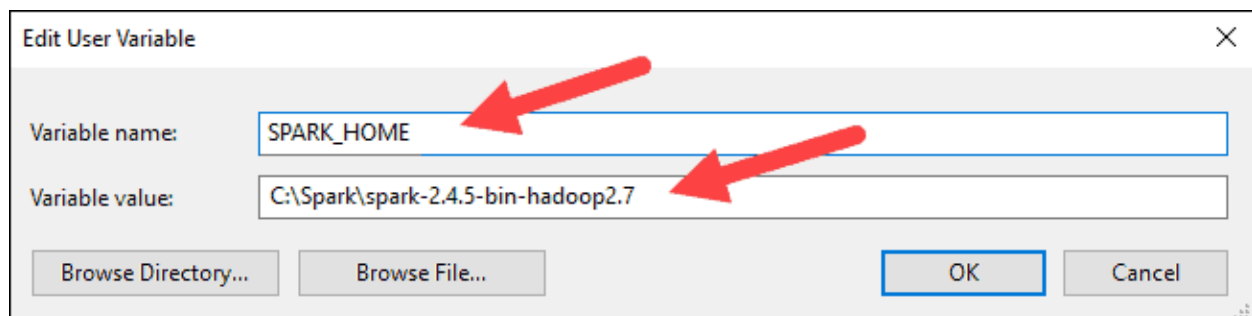
2. Select the result labeled **Edit the system environment variables**.

3. A System Properties dialog box appears. In the lower-right corner, click **Environment Variables** and then click **New** in the next window.

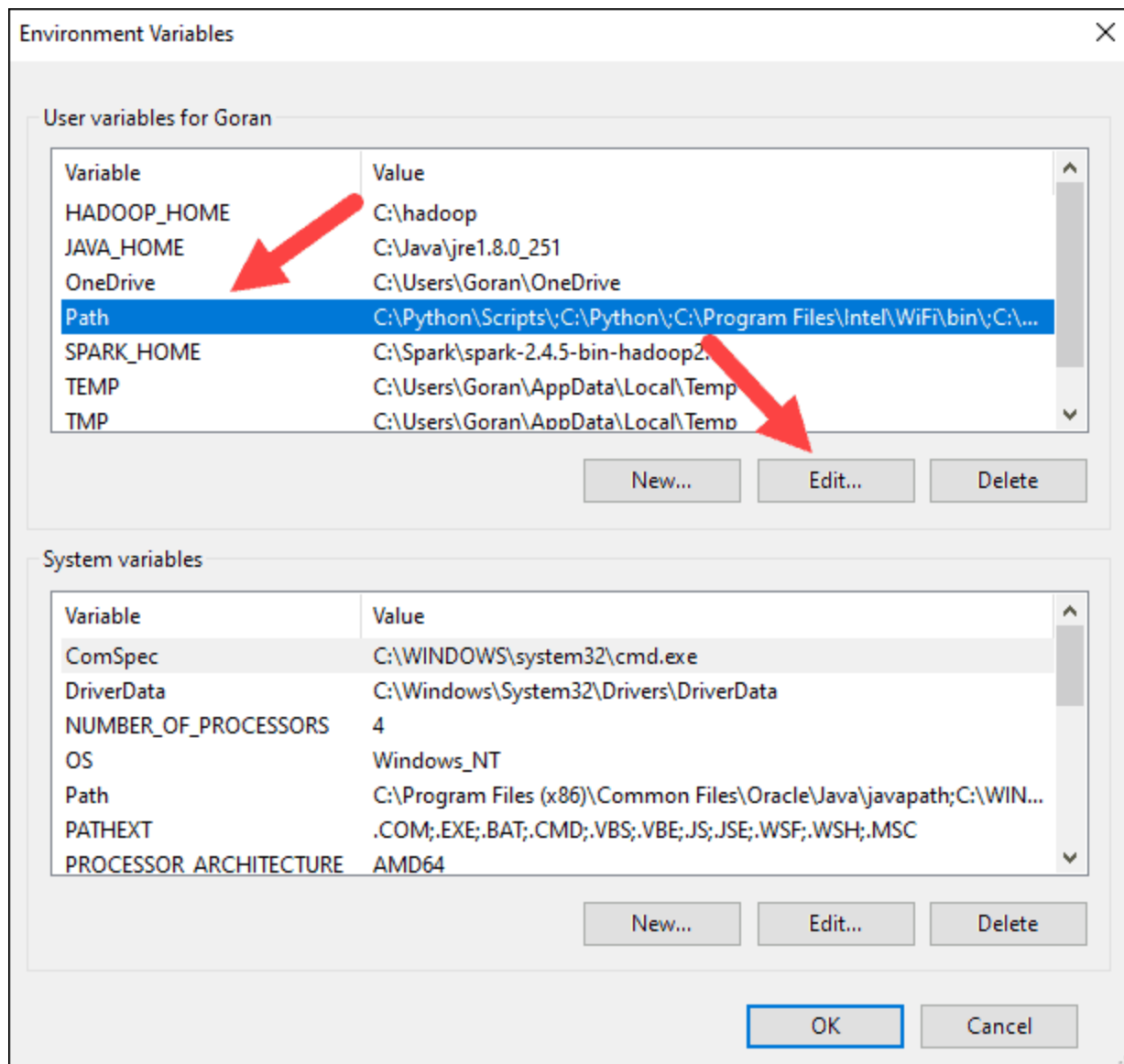


4. For *Variable Name* type **SPARK_HOME**.

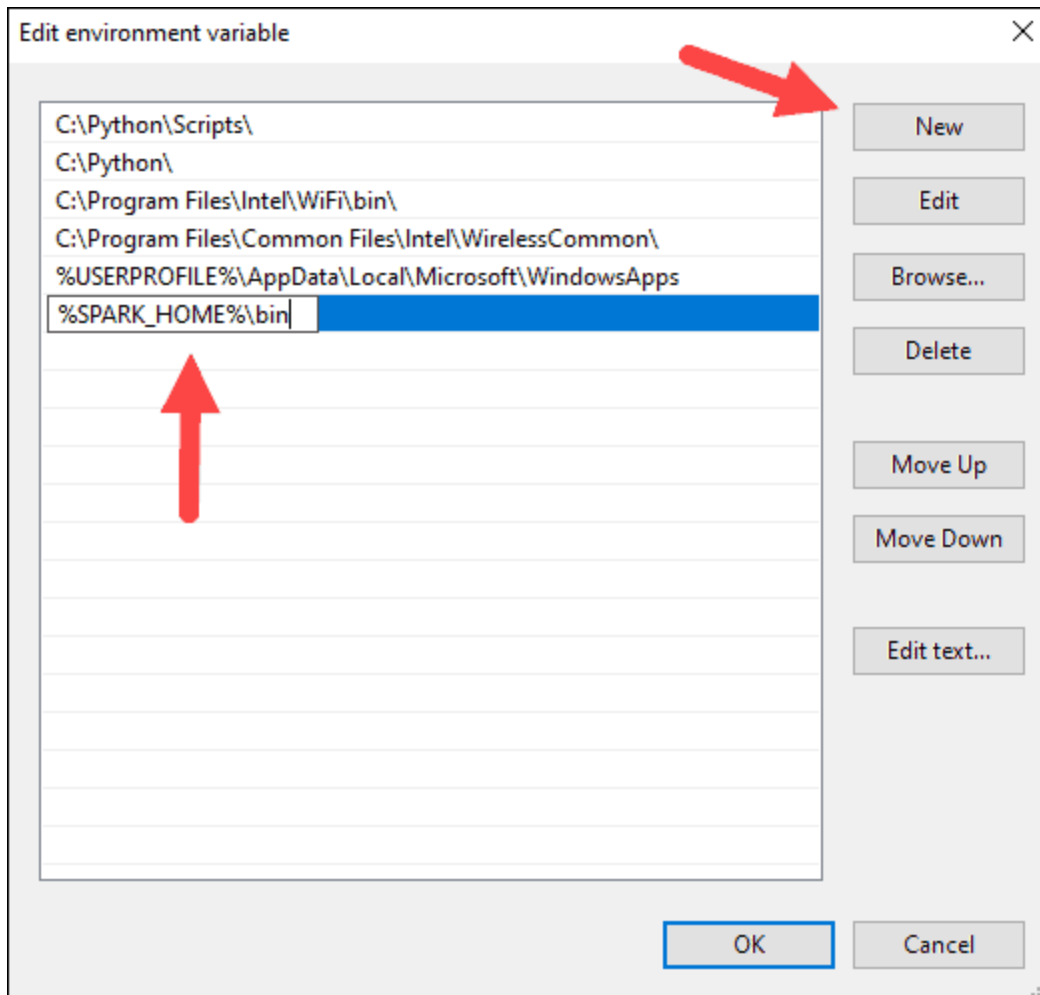
5. For *Variable Value* type **C:\Spark\spark-2.4.5-bin-hadoop2.7** and click OK. If you changed the folder path, use that one instead.



6. In the top box, click the **Path** entry, then click **Edit**. Be careful with editing the system path. Avoid deleting any entries already on the list.



7. You should see a box with entries on the left. On the right, click **New**.
8. The system highlights a new line. Enter the path to the Spark folder **C:\Spark\spark-2.4.5-bin-hadoop2.7\bin**. We recommend using **%SPARK_HOME%\bin** to avoid possible issues with the path.



9. Repeat this process for Hadoop and Java.

- For Hadoop, the variable name is **HADOOP_HOME** and for the value use the path of the folder you created earlier: **C:\hadoop**. Add **C:\hadoop\bin** to the **Path variable** field, but we recommend using **%HADOOP_HOME%\bin**.
- For Java, the variable name is **JAVA_HOME** and for the value use the path to your Java JDK directory (in our case it's **C:\Program Files\Java\jdk1.8.0_251**).

10. Click **OK** to close all open windows.

Step 6: Launch Spark

1. Open a new command-prompt window using the right-click and **Run as administrator**:

2. To start Spark, enter:

C:\Spark\spark-2.4.5-bin-hadoop2.7\bin\spark-shell

If you set the **environment path** correctly, you can type **spark-shell** to launch Spark.

3. The system should display several lines indicating the status of the application. You may get a Java pop-up. Select **Allow access** to continue.

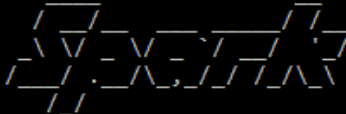
4. Finally, the Spark logo appears, and the prompt displays the **Scala shell**.

4. Open a web browser and navigate to **http://localhost:4040/**.

5. You can replace **localhost** with the name of your system.

```
C:\Spark\spark-2.4.5-bin-hadoop2.7\bin\spark-shell
```

20/05/15 16:25:38 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://DESKTOP-SFBGHOU:4040
Spark context available as 'sc' (master = local[*], app id = local-1589552754132).
Spark session available as 'spark'.
Welcome to

 version 2.4.5

Using Scala version 2.11.12 (Java HotSpot(TM) Client VM, Java 1.8.0_251)
Type in expressions to have them evaluated.
Type :help for more information.

scala> 

6. You should see an Apache Spark shell Web UI. The example below shows the *Executors* page.


2.4.5

[Jobs](#)
[Stages](#)
[Storage](#)
[Environment](#)
[Executors](#)

Spark shell application U

Executors

[Show Additional Metrics](#)

Summary

	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks
Active(1)	0	0.0 B / 434 MB	0.0 B	4	0	0	0	0
Dead(0)	0	0.0 B / 0.0 B	0.0 B	0	0	0	0	0
Total(1)	0	0.0 B / 434 MB	0.0 B	4	0	0	0	0

Executors

Show 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Completed Tasks
driver	DESKTOP-SFBGHOU:61547	Active	0	0.0 B / 434 MB	0.0 B	4	0	0	0

Showing 1 to 1 of 1 entries

[Previous](#)
[1](#)
[Next](#)

7. To exit Spark and close the Scala shell, press **ctrl-d** in the command-prompt window.

Step 7: Test Spark

In this example, we will launch the Spark shell and use Scala to read the contents of a file. You can use an existing file, such as the `README` file in the Spark directory, or you can create your own. We created `pnaptest` with some text.

1. Open a command-prompt window and navigate to the folder with the file you want to use and launch the Spark shell.
2. First, state a variable to use in the Spark context with the name of the file. Remember to add the file extension if there is any.

```
val x = sc.textFile("pnaptest")
```

3. The output shows an RDD is created. Then, we can view the file contents by using this command to call an action:

```
x.take(11).foreach(println)
```

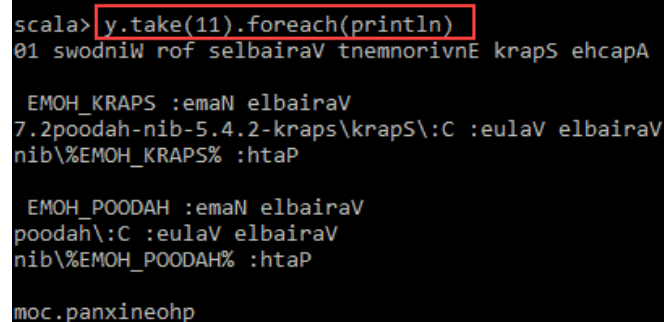
This command instructs Spark to print 11 lines from the file you specified. To perform an action on this file (**value x**), add another value **y**, and do a map transformation.

4. For example, you can print the characters in reverse with this command:

```
val y = x.map(_.reverse)
```

5. The system creates a child RDD in relation to the first one. Then, specify how many lines you want to print from the value **y**:

```
y.take(11).foreach(println)
```



```
scala> y.take(11).foreach(println)
01 swodniW rof selbairaV tnemnorivnE krapS ehcapA

EMOH_KRAPPS :emaN elbairaV
7.2poodah-nib-5.4.2-krapS\krapS\C :eulaV elbairaV
nib\%EMOH_KRAPPS% :htaP

EMOH_POODAH :emaN elbairaV
poodah\C :eulaV elbairaV
nib\%EMOH_POODAH% :htaP

moc.panxineohp
```

The output prints 11 lines of the *pnaptest* file in the reverse order.

When done, exit the shell using **ctrl-d**.

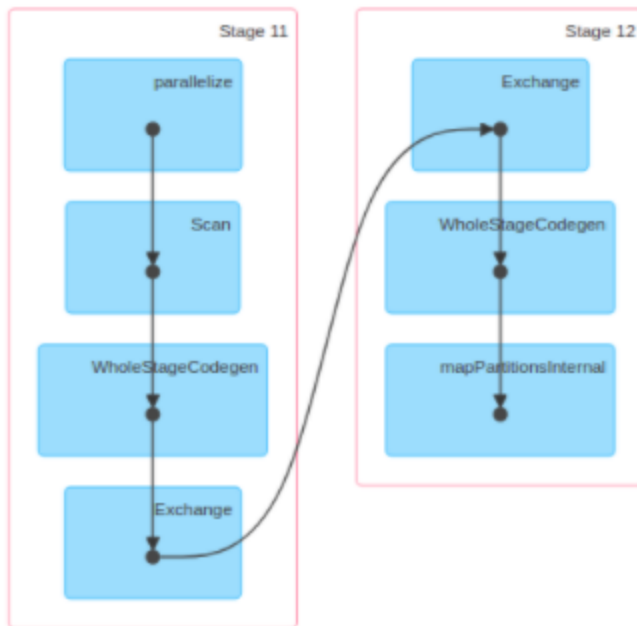
Conclusion:

You should now have a working installation of Apache Spark on Windows 10 with all dependencies installed. Get started running an instance of Spark in your Windows environment.

Q5) What is a dag in spark?

(Directed Acyclic Graph) DAG in Apache Spark is a set of Vertices and Edges, where vertices represent the RDDs and the edges represent the Operation to be applied on RDD.

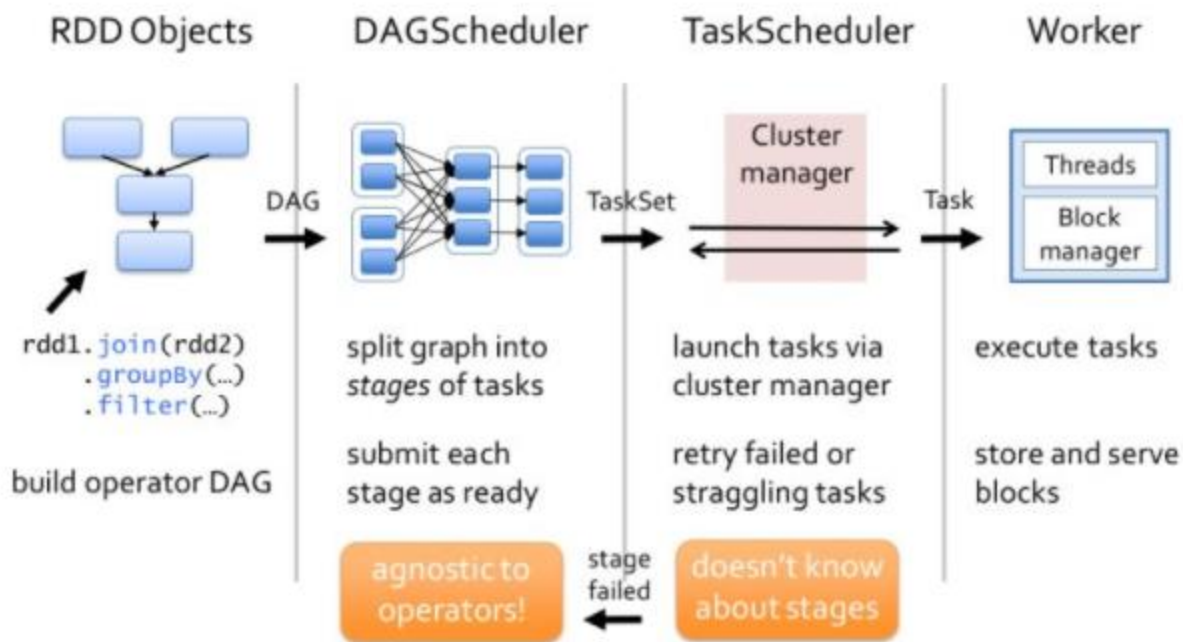
▼ DAG Visualization



In Spark DAG, every edge directs from earlier to later in the sequence. On the calling of Action, the created DAG submits to DAG Scheduler which further splits the graph into the stages of the task.

Apache Spark DAG allows the user to dive into the stage and expand on detail on any stage. In the stage view, the details of all RDDs belonging to that stage are expanded. The Scheduler splits the Spark RDD into stages based on various transformation applied. Each stage is comprised of tasks, based on the partitions of the RDD, which will perform same computation in parallel. The graph here refers to navigation, and directed and acyclic refers to how it is done.

The first layer is the interpreter, Spark uses a Scala interpreter, with some modifications.



Following points will explain its working:

1. As you enter your code in spark console (creating RDD's and applying operators), Spark creates a operator graph.
2. When the user runs an action (like collect), the Graph is submitted to a DAG Scheduler. The DAG scheduler divides operator graph into (map and reduce) stages/tasks.
3. A stage is comprised of tasks based on partitions of the input data. The DAG scheduler pipelines operators together to optimize the graph. For e.g. Many map operators can be scheduled in a single stage. This optimization is key to Sparks performance. The final result of a DAG scheduler is a set of stages.
4. The stages are passed on to the Task Scheduler. The task scheduler launches tasks via cluster manager. (Spark Standalone/Yarn/Mesos). The task scheduler doesn't know about dependencies among stages.
5. The Worker executes the tasks. A new JVM is started per job. The worker knows only about the code that is passed to it.

Q6) What is a dataframe in spark?

A DataFrame is a Dataset organized into named columns. It is conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations under the hood. DataFrames can be constructed from a wide array of sources such as: structured data files, tables in Hive, external databases, or existing RDDs. The DataFrame API is available in Scala, Java, Python, and R. In Scala and Java, a DataFrame is represented by a Dataset of Rows. In the Scala API, DataFrame is simply a type alias of Dataset[Row]. While, in Java API, users need to use Dataset<Row> to represent a DataFrame.

Some of the key features of DataFrame in Spark are:

- i. DataFrame is a distributed collection of data organized in named column. It is equivalent to the table in RDBMS.
- ii. It can deal with both structured and unstructured data formats. For e.g. Avro, CSV, elastic search, and Cassandra. It also deals with storage systems HDFS, HIVE tables, MySQL, etc.
- iv. The DataFrame API's are available in various programming languages. For e.g. Java, Scala, Python, and R.
- v. It provides Hive compatibility. We can run unmodified Hive queries on existing Hive warehouse.
- vi. It can scale from kilobytes of data on the single laptop to petabytes of data on a large cluster.

Q7) What is an action in spark?

Actions, which return a value to the driver program after running a computation on the dataset.

Transformations create RDDs from each other, but when we want to work with the actual dataset, at that point action is performed. When the action is triggered after the result, new RDD is not formed like transformation. Thus, Actions are Spark RDD operations that give non-RDD values. The values of action are stored to drivers or to the external storage system. It brings laziness of RDD into motion.

An action is one of the ways of sending data from Executer to the driver. Executors are agents that are responsible for executing a task. While the driver is a JVM process that coordinates workers and execution of the task. E.g. Count, Collect.

Q8) In which city did the first spark summit take place in 2013?

The first spark summit took place in Downtown San Francisco on Dec 2nd 2013.

Q9) Which tells spark how and where to access a cluster?

The first thing a Spark program must do is to create a SparkContext object, which tells Spark how to access a cluster. To create a SparkContext you first need to build a SparkConf object that contains information about your application.

Only one SparkContext may be active per JVM. You must stop() the active SparkContext before creating a new one.

```
val conf = new SparkConf().setAppName(appName).setMaster(master)

new SparkContext(conf)
```

The appName parameter is a name for your application to show on the cluster UI. master is a Spark, Mesos or YARN cluster URL, or a special “local” string to run in local mode. In practice, when running on a cluster, you will not want to hardcode master in the program, but rather launch the application with spark-submit and receive it there. However, for local testing and unit tests, you can pass “local” to run Spark in-process.

Q10) What is an accumulator in spark?

There are two main abstractions in spark:

1. RDD's: collection of elements partitioned across the nodes of the cluster that can be operated on in parallel.
2. Shared Variables: shared variables that can be used in parallel operations

By default, when Spark runs a function in parallel as a set of tasks on different nodes, it ships a copy of each variable used in the function to each task. Sometimes, a variable needs to be shared across tasks, or between tasks and the driver program. Spark supports two types of shared **variables**: *broadcast variables*, which can be used to cache a value in memory on all nodes, and **accumulators**, which are variables that are only “added” to, such as counters and sums.

Accumulators in Spark are used specifically to provide a mechanism for safely updating a variable when execution is split up across worker nodes in a cluster.

Accumulators are variables that are only “added” to through an associative and commutative operation and can therefore be efficiently supported in parallel. They can

be used to implement counters (as in MapReduce) or sums. Spark natively supports accumulators of numeric types, and programmers can add support for new types.

As a user, you can create named or unnamed accumulators. As seen in the image below, a named accumulator (in this instance counter) will display in the web UI for the stage that modifies that accumulator. Spark displays the value for each accumulator modified by a task in the “Tasks” table.

Accumulators in the Spark UI

Tracking accumulators in the UI can be useful for understanding the progress of running stages (NOTE: this is not yet supported in Python).

A numeric accumulator can be created by calling `SparkContext.longAccumulator()` or `SparkContext.doubleAccumulator()` to accumulate values of type `Long` or `Double`, respectively. Tasks running on a cluster can then add to it using the `add` method. However, they cannot read its value. Only the driver program can read the accumulator’s value, using its `value` method.

The code below shows an accumulator being used to add up the elements of an array:

```
scala> val accum = sc.longAccumulator("My Accumulator")
```

```
accum: org.apache.spark.util.LongAccumulator = LongAccumulator(id: 0, name: Some(My Accumulator), value: 0)
```

```
scala> sc.parallelize(Array(1, 2, 3, 4)).foreach(x => accum.add(x))
```

```
scala> accum.value
```

```
res2: Long = 10
```

While this code used the built-in support for accumulators of type `Long`, programmers can also create their own types by subclassing `AccumulatorV2`. The `AccumulatorV2` abstract class has several methods which one has to override: `reset` for resetting the accumulator to zero, `add` for adding another value into the accumulator, `merge` for merging another same-type accumulator into this one. Other methods that must be

overridden are contained in the API documentation. For example, supposing we had a `MyVector` class representing mathematical vectors, we could write:

```
class VectorAccumulatorV2 extends AccumulatorV2[MyVector, MyVector] {
```

```
    private val myVector: MyVector = MyVector.createZeroVector
```

```
    def reset(): Unit = {
        myVector.reset()
    }
```

```
    def add(v: MyVector): Unit = {
        myVector.add(v)
    }
    ...
}
```

// Then, create an Accumulator of this type:

```
val myVectorAcc = new VectorAccumulatorV2
```

// Then, register it into spark context:

```
sc.register(myVectorAcc, "MyVectorAcc1")
```

Note that, when programmers define their own type of `AccumulatorV2`, the resulting type can be different than that of the elements added.

For accumulator updates performed inside actions only, Spark guarantees that each task's update to the accumulator will only be applied once, i.e. restarted tasks will not

update the value. In transformations, users should be aware of that each task's update may be applied more than once if tasks or job stages are re-executed.

Accumulators do not change the lazy evaluation model of Spark. If they are being updated within an operation on an RDD, their value is only updated once that RDD is computed as part of an action. Consequently, accumulator updates are not guaranteed to be executed when made within a lazy transformation like `map()`.

Important FAQs

How do I prepare for a Spark Interview?

You can start by going through various blogs available online that provide you with important questions. These questions will help you gain confidence in some of the commonly asked questions. You can also read blogs on Apache Spark provided by sites like Great Learning that will help you brush up your knowledge.

What is Spark good for?

Spark is good for training machine learning algorithms, stream processing, data integration, and interactive analytics.

Is Spark hard to learn?

Learning Spark is not difficult if you have a brief understanding of python and other programming languages. The APIs are provided in Python, Java, and Scala. You can take up the Apache Spark course on Great Learning and start learning.

Why Spark is faster than MapReduce?

Spark uses RDDs, i.e., Resilient Distributed Datasets which support multiple map operations in the memory. MapReduce has to write down interim results to the disk. Hence, Spark is faster.

What is a Spark core?

Spark Core is the fundamental unit of the entire Spark project which provides all sorts of functionalities like scheduling, task-dispatching, and I/O operations, etc.

When should you not use Spark?

You should not use Spark if you want query per/sec to get data to put on the website to end-users. As for each request, Spark will load the file data to search for the one single record in it.

What is Spark SQL?

Spark SQL streamlines the process of querying data stored both in external sources and RDDs (Spark's distributed datasets). Spark SQL effectively blurs the lines between relational tables and RDDs.

What is RDD in spark?

RDD in Spark is Resilient Distributed Datasets; it enables Spark to have multiple map operations in the memory, making it very fast.

Broadcast Hash Join

Suitable for scenarios where one of the datasets is small enough to be broadcasted to all executor nodes. The smaller dataset is broadcasted to all nodes in the cluster, and then each node joins the broadcasted dataset with partitions of the larger dataset locally. Efficient when one dataset is significantly smaller than the other.

Syntax:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import broadcast

spark =
SparkSession.builder.appName("JoinStrategies").getOrCreate()

smallDF = spark.read.csv("small_dataset.csv", header=True)
largeDF = spark.read.csv("large_dataset.csv", header=True)

joinedDF = largeDF.join(broadcast(smallDF), "key")
joinedDF.show()
```

Sort Merge Join

Default join strategy for large datasets that do not fit into memory. Both datasets are sorted by the join key and then merged. This requires a shuffle to sort the data, which can be expensive. Suitable for large datasets where both tables are too large to broadcast.

Syntax:

```
df1 = spark.read.csv("dataset1.csv", header=True)
df2 = spark.read.csv("dataset2.csv", header=True)
```



```
joinedDF = df1.join(df2, "key")  
joinedDF.show()
```

Shuffle Hash Join

Used when the datasets are too large for a broadcast join, but one of the datasets is still relatively small. The smaller dataset is hashed and the data is partitioned based on the hash of the join keys. Each partition of the larger dataset is then joined with the corresponding partition of the smaller dataset. Efficient when one dataset is smaller but not small enough to broadcast.

Syntax:

```
df1 = spark.read.csv("large_dataset.csv", header=True)  
df2 = spark.read.csv("medium_dataset.csv", header=True)  
joinedDF = df1.hint("SHUFFLE_HASH").join(df2, "key")  
joinedDF.show()
```

Broadcast Nested Loop Join

Used when no other join strategy is applicable, often as a fallback method. A Cartesian product of the datasets is produced, and then a filter is applied to select the matching rows. It broadcasts the smaller table and then iterates through it for each row of the larger table. Applicable in cross joins or when join conditions are complex and other joins are not feasible.

Syntax:

```
df1 = spark.read.csv("dataset1.csv", header=True)  
df2 = spark.read.csv("dataset2.csv", header=True)
```

```
joinedDF = df1.crossJoin(df2)
joinedDF.show()
```

Cartesian Product Join

Computes the Cartesian product of two datasets. Every row of the first dataset is joined with every row of the second dataset. Used explicitly for cross joins or when the join condition is not specified.

Syntax:

```
df1 = spark.read.csv("dataset1.csv", header=True)
df2 = spark.read.csv("dataset2.csv", header=True)
joinedDF = df1.crossJoin(df2)
joinedDF.show()
```

Skew Join

Special strategy to handle data skew in join keys. Skewed keys are detected, and their corresponding partitions are split into smaller partitions to distribute the load evenly across the cluster. Useful when certain keys in the join have disproportionately large amounts of data.

Syntax:

```
spark.conf.set("spark.sql.auto.skewJoin.enabled", "true")
df1 = spark.read.csv("large_skewed_dataset.csv", header=True)
df2 = spark.read.csv("large_skewed_dataset.csv", header=True)
joinedDF = df1.join(df2, "key")
joinedDF.show()
```


Data Cleaning with Apache Spark

Spark Schema

```
.printSchema()
```

```
# Import schema
```

```
import pyspark.sql.types
```

```
from pyspark.sql.types import *
```

```
peopleSchema = StructType([
```

```
# Define the name field
```

```
StructField('name', StringType(), True), # Define the name field
```

```
StructField('age', IntegerType(), True), # Add the age field
```

```
StructField('city', StringType(), True) ]) # Add the city field
```

```
#if false, means cannot be nullable.
```

```
people_df = spark.read.format('csv').load(name='rawdata.csv',  
schema=peopleSchema)
```

Immutability and Lazy Processing

A component of functional programming

Defined once

Unable to be directly modified

Re-created if reassigned

Able to be shared efficiently

```
# Immutability Example
# Load the CSV file
aa_dfw_df = spark.read.format('csv').options(Header=True).load('AA_DFW_2018.csv')

# Add the airport column using the F.lower() method
aa_dfw_df = aa_dfw_df.withColumn('airport', F.lower(aa_dfw_df['Destination Airport']))

# Drop the Destination Airport column
aa_dfw_df = aa_dfw_df.drop(aa_dfw_df['Destination Airport'])

# Show the DataFrame
aa_dfw_df.show()
```

Understanding Parquet

Difficulties with CSV files

No defined schema (no data type, no column name beyond a header row)

Nested data requires special (content containing a comma requires escaping, using the escape character within content requires even further escaping)

handling Encoding format limited

for spark: slow to parse, cannot be shared during the import process; if no schema is defined, all data must be read before a schema can be inferred, forcing the code to read the file twice.

for spark: files cannot be filtered (no 'predicate pushdown', ordering tasks to do the least amount of work, filtering data prior to processing is one of the primary optimizations of predicate pushdown.)

for spark: any intermediate use requires redefining schema. Spark process are often multi-step and may utilize an intermediate file representation. These representations allow data to be used later without regenerating the data from source.

The Parquet Format

columnar data format

supported in spark and other data processing frameworks

supports predicate pushdown

automatically stores schema information

binary file format

Parquet is a compressed columnar data format developed for use in any Hadoop based system. Include: Spark, Hadoop, Apache Impala... Perfect for intermediary or on-disk representation of processed data.

predicate pushdown This means Spark will only process the data necessary to complete the operations you define versus reading the entire dataset.

Working with Parquet

```
df = spark.read.format('parquet').load('filename.parquet')
df = spark.read.parquet('filename.parquet')
```

```
df.write.format('parquet').save('filename.parquet')
df.write.parquet('filename.parquet') #mode='overwrite'
```

Parquet with SQL

parquet - dataframe - table

```
flight_df = spark.read.parquet('flights.parquet')
flight_df.createOrReplaceTempView('flights')
short_flights_df = spark.sql('SELECT * FROM flights WHERE flightduration <100')

# Run a SQL query of the average flight duration
avg_duration = spark.sql('SELECT avg(flight_duration) from flights').collect()[0]
print('The average flight time is: %d' % avg_duration)
```

DataFrame column operations

DataFrames:

Made up of rows & columns

Immutable

Use various transformation operations to modify data

```
# Return rows where name starts with "M"
voter_df.filter(voter_df.name.like('M%'))
# Return name and position only
voters = voter_df.select('name', 'position')
```

Common DataFrame transformations

#Filter / Where interchangeable

```
voter_df.filter(voter_df.date > '1/1/2019') # or voter_df.where(...)
```

#Select

```
voter_df.select(voter_df.name)
```

#withColumn creates new column

```
voter_df.withColumn('year', voter_df.date.year)
```

#drop

```
voter_df.drop('unused_column')
```

Filtering data

Remove nulls

Remove odd entries

Split data from combined sources

Negate with ~

```
voter_df.filter(voter_df['name'].isNotNull())  
voter_df.filter(voter_df.date.year > 1800)  
voter_df.where(voter_df['_c0'].contains('VOTE'))  
voter_df.where(~ voter_df._c1.isNull())
```

Show the distinct VOTER_NAME entries

```
voter_df.select('VOTER_NAME').distinct().show(40, truncate=False)
```

Filter voter_df where the VOTER_NAME is 1-20 characters in length

```
voter_df = voter_df.filter('length(VOTER_NAME) > 0 and length(VOTER_NAME) < 20')
```

Filter out voter_df where the VOTER_NAME contains an underscore

```
voter_df = voter_df.filter(~ F.col('VOTER_NAME').contains('_'))
```

Show the distinct VOTER_NAME entries again

```
voter_df.select('VOTER_NAME').distinct().show(40, truncate=False)
```

Column string transformations

```
#Contained in pyspark.sql.functions
import pyspark.sql.functions as F

#Applied per column as transformation
voter_df.withColumn('upper', F.upper('name'))

#Can create intermediary columns
voter_df.withColumn('splits', F.split('name', ' '))

#Can cast to other types
voter_df.withColumn('year', voter_df['c4'].cast(IntegerType()))
```

ArrayType() column functions

Various utility functions / transformations to interact with ArrayType() .

.size() - returns length of arrayType() column

.getItem() - used to retrieve a specific item at index of list column.

```
# Add a new column called splits separated on whitespace
voter_df = voter_df.withColumn('splits', F.split(voter_df.VOTER_NAME, '\s+'))

# Create a new column called first_name based on the first item in splits
voter_df = voter_df.withColumn('first_name', voter_df.splits.getItem(0))

# Get the last entry of the splits list and create a column called last_name
voter_df = voter_df.withColumn('last_name', voter_df.splits.getItem(F.size('spl

# Drop the splits column
voter_df = voter_df.drop('splits')

# Show the voter_df DataFrame
voter_df.show()
```

Conditional DataFrame column operations

Conditional clauses

Conditional Clauses are:

Inline version of if / then / else

```
.when()  
.otherwise()
```

```
df.select(df.Name, df.Age,  
.when(df.Age >= 18, "Adult")  
.when(df.Age < 18, "Minor"))
```

```
df.select(df.Name, df.Age,  
.when(df.Age >= 18, "Adult")  
.otherwise("Minor"))
```

```
# Add a column to voter_df for any voter with the title **Councilmember**  
voter_df = voter_df.withColumn('random_val',  
    when(voter_df.TITLE == 'Councilmember', F.rand()))
```

```
# Add a column to voter_df for a voter based on their position  
voter_df = voter_df.withColumn('random_val',  
    when(voter_df.TITLE == 'Councilmember', F.rand())  
    .when(voter_df.TITLE == 'Mayor', 2)  
    .otherwise(0))
```

User defined functions

Python method

Wrapped via the `pyspark.sql.functions.udf` method

Stored as a variable

Called like a normal Spark function

```
def getFirstAndMiddle(names):
    # Return a space separated string of names
    return ' '.join(names[:-1])

# Define the method as a UDF
udfFirstAndMiddle = F.udf(getFirstAndMiddle, StringType())

# Create a new column using your UDF
voter_df = voter_df.withColumn('first_and_middle_name', udfFirstAndMiddle(voter_

# Drop the unnecessary columns then show the DataFrame
voter_df = voter_df.drop('first_name')
voter_df = voter_df.drop('splits')
voter_df.show()
```

Reverse string UDF

```
#Define a Python method
def reverseString(mystr):
    return mystr[::-1]

#Wrap the function and store as a variable
udfReverseString = udf(reverseString, StringType())

#Use with Spark
user_df = user_df.withColumn('ReverseName', udfReverseString())
```

Argument-less example

```
def sortingCap():
    return random.choice(['G', 'H', 'R', 'S'])
udfSortingCap = udf(sortingCap, StringType())
user_df = user_df.withColumn('Class', udfSortingCap())
```

Partitioning and lazy processing

Partitioning

DataFrames are broken up into partitions

Partition size can vary

Each partition is handled independently

To check the number of partitions, use the method `.rdd.getNumPartitions()` on a `DataFrame`.

Lazy processing

Transformations are lazy

- * `.withColumn(...)`

- * `.select(...)`

- * `.cache()`

Nothing is actually done until an action is performed

- * `.count()`

- * `.write(...)`

- * `.show()`

Transformations can be re-ordered for best performance

Sometimes causes unexpected behavior

Adding IDs

Normal ID fields:

Common in relational databases

Most usually an integer increasing, sequential and unique

Not very parallel

Monotonically increasing IDs

```
pyspark.sql.functions.monotonically_increasing_id()
```

Integer (64-bit), increases in value, unique

Not necessarily sequential (gaps exist)

Completely parallel

```
# Select all the unique council voters
voter_df = df.select(df["VOTER NAME"]).distinct()

# Count the rows in voter_df
print("\nThere are %d rows in the voter_df DataFrame.\n" % voter_df.count())

# Add a ROW_ID
voter_df = voter_df.withColumn('ROW_ID', F.monotonically_increasing_id())

# Show the rows with 10 highest IDs in the set
voter_df.orderBy(voter_df.ROW_ID.desc()).show(10)
```

More ID Tricks

Depending on your needs, you may want to start your IDs at a certain value so there isn't overlap with previous runs of the Spark task. This behavior is similar to how IDs would behave in a relational database. Make sure that the IDs output from a monthly Spark task start at the highest value from the previous month.

Caching

Caching in Spark:

Stores DataFrames in memory or on disk

Improves speed on later transformations / actions

Reduces resource usage

Disadvantages of caching

Very large data sets may not fit in memory

Local disk based caching may not be a performance improvement

Cached objects may not be available

Caching tips When developing Spark tasks:

Cache only if you need it

Try caching DataFrames at various points and determine if your performance improves

Cache in memory and fast SSD / NVMe storage

Cache to slow local disk if needed

Use intermediate files!

Stop caching objects when finished

Eviction Policy Least Recently Used (LRU)

Caching is a lazy operation. It requires an action to trigger it. eg.

```
spark.sql("select count(*) from text").show()
```

```
partitioned_df.count()
```

```
df.cache()      #df.persist() df.persist(storageLevel=pyspark.StorageLevel.MEMORY_
df.unpersist()
```

#Determining whether a dataframe is cached

```
df.is_cached
```

#storage level useDisk useMemory useOffHeap deserialized replication

```
df.storageLevel
```

#Caching a table

```
df.createOrReplaceTempView('df')
```

```
spark.catalog.cacheTable('df')
```

```
spark.catalog.isCached(tableName='df')
```

```
spark.catalog.dropTempView('table1')
```

List the tables

```
print("Tables:\n", spark.catalog.listTables())
```

Uncaching a table

```
spark.catalog.uncacheTable('df')
```

```
spark.catalog.clearCache()
```

Improve import performance

Spark clusters

Spark Clusters are made of two types of processes

Driver process

Worker processes

Import performance

Important parameters:

Number of objects (Files, Network locations, etc)

More objects better than larger ones

Can import via wildcard

```
airport_df = spark.read.csv('airports-*.txt.gz')
```

General size of objects

Spark performs better if objects are of similar size

It's safe to assume the more import objects available, the better the cluster can divvy up the job.

Schemas

A well-dened schema will drastically improve import performance

Avoids reading the data multiple times

Provides validation on import

How to split objects

Use OS utilities / scripts (split, cut, awk)

```
split -l 10000 -d largefile chunk-
```

每个文件100000行, 字符, 名字叫largefile, 生成chunk0000开始

Use custom scripts

Write out to Parquet

```
df_csv = spark.read.csv('singlelargefile.csv')
```

```
df_csv.write.parquet('data.parquet')
df = spark.read.parquet('data.parquet')
```

Explaining the Spark execution plan

```
voter_df = df.select(df['VOTER NAME']).distinct()
voter_df.explain()
```

== Physical Plan ==

* (2) HashAggregate(keys=[VOTER NAME#15], functions=[]) +- Exchange
hashpartitioning(VOTER NAME#15, 200)

+ * (1) HashAggregate(keys=[VOTER NAME#15], functions=[])

+ * (1) FileScan csv [VOTER NAME#15] Batched: false, Format: CSV, Location:
InMemoryFileIndex[file:/DallasCouncilVotes.csv.gz],
PartitionFilters: [], PushedFilters: [],
ReadSchema: struct<VOTER NAME:string>

Shuffling

Shuffling refers to moving data around to various workers to complete a task

Hides complexity from the user

Can be slow to complete

Lowers overall throughput

Is often necessary, but try to minimize

How to limit shuffling

Limit use of `.repartition(num_partitions)`

Use `.coalesce(num_partitions)` instead

Use care when calling `.join()` Use `.broadcast()`

May not need to limit it

Broadcasting

Provides a copy of an object to each worker

Prevents undue / excess communication between nodes

Can drastically speed up `.join()` operations

A couple tips:

Broadcast the smaller DataFrame. The larger the DataFrame, the more time required to transfer to the worker nodes.

On small DataFrames, it may be better skip broadcasting and let Spark figure out any optimization on its own.

If you look at the query execution plan, a `broadcastHashJoin` indicates you've successfully configured broadcasting.

Use the `.broadcast(<DataFrame>)` method

```
from pyspark.sql.functions import broadcast
combined_df = df_1.join(broadcast(df_2))
```

Cluster Configurations

Configuration options

Spark contains many configuration settings

These can be modied to match needs

Reading configuration settings: `spark.conf.get(<configuration name>)`

Writing configuration settings `spark.conf.set(<configuration name>)`

Configure Spark to use 500 partitions

```
spark.conf.set('spark.sql.shuffle.partitions', 500)
```



```
# Name of the Spark application instance
app_name = spark.conf.get('spark.app.name')

# Driver TCP port
driver_tcp_port = spark.conf.get('spark.driver.port')

# Number of join partitions
num_partitions = spark.conf.get('spark.sql.shuffle.partitions')

# Store the number of partitions in variable
before = departures_df.rdd.getNumPartitions()

# Configure Spark to use 500 partitions
spark.conf.set('spark.sql.shuffle.partitions', 500)

# Recreate the DataFrame using the departures data file
departures_df = spark.read.csv('departures.txt.gz').distinct()
```

Cluster Types

Spark deployment options:

Single node (deploying all components on a single system, can be physical/VM/container)

Standalone (dedicated machines as the driver and workers)

Managed (cluster components are handled by a third party cluster manager)

YARN

Mesos

Kubernetes

Driver

Task assignment

Result consolidation

Shared data access

Tips:

Driver node should have double the memory of the worker

Fast local storage helpful

Worker

Runs actual tasks

Ideally has all code, data, and resources for a given task

Recommendations:

More worker nodes is often better than larger workers

Test to find the balance

Fast local storage extremely useful

Data Pipelines

Input(s)

CSV, JSON, web services, databases

Transformations

```
withColumn() ,  
.filter() ,  
.drop()
```

Output(s)

CSV, Parquet, database Validation

Analysis

Pipeline details

Not formally defined in Spark

Typically all normal Spark code required for task

```
schema = StructType([
    StructField('name', StringType(), False),
    StructField('age', StringType(), False)
])
df = spark.read.format('csv').load('datafile').schema(schema) df = df.withColumn
...
df.write.parquet('outdata.parquet')
df.write.json('outdata.json')
```

```
# Import the data to a DataFrame
departures_df = spark.read.csv('2015-departures.csv.gz', header=True)

# Remove any duration of 0
departures_df = departures_df.filter(~departures_df[3].contains('0'))

# Add an ID column
departures_df = departures_df.withColumn('id', F.monotonically_increasing_id())

# Write the file out to JSON format
departures_df.write.json('output.json')
```

Data Handling

Parsing

Incorrect data

- * Empty rows
- * Commented lines
- * Headers

Nested structures

- * Multiple delimiters

Non-regular data

- * Differing numbers of columns per row

Removing blank lines, headers, and comments

Spark's CSV parser:

Automatically removes blank lines

Can remove comments using an optional argument

```
df1 = spark.read.csv('datafile.csv.gz', comment='#')
```

Handles header fields

Defined via argument

Ignored if a schema is defined

```
df1 = spark.read.csv('datafile.csv.gz', header='True')
```

Count the number of rows beginning with '#'

```
comment_count =  
annotations_df.where(col('_c0').startswith('#')).count()  
  
# Split _c0 on the tab character and store the list in a variable  
tmp_fields = F.split(annotations_df['_c0'], '\t')  
  
# Create the colcount column on the DataFrame  
annotations_df = annotations_df.withColumn('colcount', F.size(tmp_fields))  
  
# Remove any rows containing fewer than 5 fields  
annotations_df_filtered = annotations_df.filter(~ (annotations_df["colcount"] <  
  
# Count the number of rows  
final_count = annotations_df_filtered.count()  
print("Initial count: %d\nFinal count: %d" % (initial_count, final_count))
```

Automatic column creation

Spark will:

Automatically create columns in a DataFrame based on sep argument `df1 = spark.read.csv('datafile.csv.gz', sep=',')`

Defaults to using `,`

Can still successfully parse if sep is not in string

```
df1 = spark.read.csv('datafile.csv.gz', sep='*')
```

Stores data in column defaulting to `_c0`

Allows you to properly handle nested separators

```
# Split the content of _c0 on the tab character (aka, '\t')
split_cols = F.split(annotations_df['_c0'], '\t')

# Add the columns folder, filename, width, and height
split_df = annotations_df.withColumn('folder', split_cols.getItem(0))
split_df = split_df.withColumn('filename', split_cols.getItem(1))
split_df = split_df.withColumn('width', split_cols.getItem(2))
split_df = split_df.withColumn('height', split_cols.getItem(3))

def retriever(cols, colcount):
    # Return a list of dog data
    return cols[4:colcount]

# Define the method as a UDF
udfRetriever = F.udf(retriever, ArrayType(StringType()))

# Create a new column using your UDF
split_df = split_df.withColumn('dog_list', udfRetriever(split_df.split_cols, split_df.colcount))

# Remove the original column, split_cols, and the colcount
split_df = split_df.drop('_c0').drop('colcount').drop('split_cols')
```

Data Validation

Validation is:

Verifying that a dataset complies with the expected format

Number of rows / columns

Data types

Complex validation rules

Validating via joins

Compares data against known values

Easy to find data in a given set

Comparatively fast

```
parsed_df = spark.read.parquet('parsed_data.parquet')
company_df = spark.read.parquet('companies.parquet')
verified_df = parsed_df.join(company_df, parsed_df.company == company_df.compan\

# Rename the column in valid_folders_df
valid_folders_df = valid_folders_df.withColumnRenamed('_c0', 'folder')

# Count the number of rows in split_df
split_count = split_df.count()

# Join the DataFrames
joined_df = split_df.join(F.broadcast(valid_folders_df), "folder")

# Compare the number of rows remaining
joined_count = joined_df.count()
print("Before: %d\nAfter: %d" % (split_count, joined_count))
```

This *automatically* removes any rows with a company not in the valid_df !

Complex rule validation

Using Spark components to validate logic:

Calculations

Verifying against external source

Likely uses a UDF to modify / verify the DataFrame

```
# Determine the row counts for each DataFrame
split_count = split_df.count()
joined_count = joined_df.count()

# Create a DataFrame containing the invalid rows
invalid_df = split_df.join(F.broadcast(joined_df), 'folder', 'left_anti')

# Validate the count of the new DataFrame is as expected
invalid_count = invalid_df.count()
print(" split_df:\t%d\n joined_df:\t%d\n invalid_df: \t%d" % (split_count, join\

# Determine the number of distinct folder columns removed
invalid_folder_count = invalid_df.select('folder').distinct().count()
print("%d distinct invalid folders found" % invalid_folder_count)
```

```
# Select the dog details and show 10 untruncated rows
print(joined_df.select('dog_list').show(truncate=False))

# Define a schema type for the details in the dog list
DogType = StructType([
    StructField("breed", StringType(), False),
    StructField("start_x", IntegerType(), False),
    StructField("start_y", IntegerType(), False),
    StructField("end_x", IntegerType(), False),
    StructField("end_y", IntegerType(), False)
])

# Create a function to return the number and type of dogs as a tuple
def dogParse(doglist):
    dogs = []
    for dog in doglist:
        (breed, start_x, start_y, end_x, end_y) = dog.split(',')
        dogs.append((breed, int(start_x), int(start_y), int(end_x), int(end_y)))
    return dogs

# Create a UDF
udfDogParse = F.udf(dogParse, ArrayType(DogType))

# Use the UDF to list of dogs and drop the old column
joined_df = joined_df.withColumn('dogs', udfDogParse('dog_list')).drop('dog_list')

# Show the number of dogs in the first 10 rows
joined_df.select(F.size('dogs')).show(10)
```

```
# Define a UDF to determine the number of pixels per image
def dogPixelCount(doglist):
    totalpixels = 0
    for dog in doglist:
        totalpixels += (dog[3] - dog[1]) * (dog[4] - dog[2])
    return totalpixels

# Define a UDF for the pixel count
udfDogPixelCount = F.udf(dogPixelCount, IntegerType())
joined_df = joined_df.withColumn('dog_pixels', udfDogPixelCount('dogs'))

# Create a column representing the percentage of pixels
joined_df = joined_df.withColumn('dog_percent', (joined_df.dog_pixels / (joined_

# Show the first 10 annotations with more than 60% dog
joined_df.where('dog_percent > 60').show(10)
```


PRACTICE INTERVIEW QUESTIONS ON APACHE SPARK ,PYSPARK FOR DATA ENGINEERS

SET OF 82 QUESTIONS

1. How is Apache Spark different from MapReduce?

Ans:

Apache Spark	MapReduce
Spark processes data in batches as well as in real-time	MapReduce processes data in batches only
Spark runs almost 100 times faster than Hadoop MapReduce	Hadoop MapReduce is slower when it comes to large scale processing
Spark stores data in the RAM i.e. in-memory. So, it is easier to retrieve it	Hadoop MapReduce data is stored in HDFS and hence longer time to retrieve the data
Spark provides caching and in-memory data storage	Hadoop is highly disk-dependent

2. What are the important components of the Spark ecosystem?

Ans:

Apache Spark has 3 main categories that comprise its ecosystem. Those are:

- Language support: Spark can integrate with different languages to applications and perform analytics. These languages are Java, Python, Scala, and R.
- Core Components: Spark supports 5 main core components. There are Spark Core, Spark SQL, Spark Streaming, Spark MLlib, and GraphX.
- Cluster Management: Spark can be run in 3 environments. Those are the Standalone cluster, Apache Mesos, and YARN.

3. What are the different cluster managers available in Apache Spark?

Ans:

- Standalone Mode: By default, applications submitted to the standalone mode cluster will run in FIFO order, and each application will try to use all available nodes. You can launch a standalone cluster either manually, by starting a master and workers by hand or use our provided launch scripts. It is also possible to run these daemons on a single machine for testing.
- Apache Mesos: Apache Mesos is an open-source project to manage computer clusters, and can also run Hadoop applications. The advantages of deploying Spark with Mesos include dynamic partitioning between Spark and other frameworks as well as scalable partitioning between multiple instances of Spark.
- Hadoop YARN: Apache YARN is the cluster resource manager of Hadoop 2. Spark can be run on YARN as well.
- Kubernetes: Kubernetes is an open-source system for automating deployment, scaling, and management of containerized applications.

4. What is a lazy evaluation in Spark?

Ans:

When Spark operates on any dataset, it remembers the instructions. When a transformation such as a `map()` is called on an RDD, the operation is not performed instantly. Transformations in Spark are not evaluated until you perform an action, which aids in optimizing the overall data processing workflow, known as lazy evaluation.

5. What makes Spark good at low latency workloads like graph processing and Machine Learning?

Ans:

Apache Spark stores data in-memory for faster processing and building machine learning models. Machine Learning algorithms require multiple iterations and different conceptual steps to create an optimal model. Graph algorithms traverse through all the nodes and edges to generate a graph. These low latency workloads that need multiple iterations can lead to increased performance.

6. How can you connect Spark to Apache Mesos?

Ans:

There are a total of 4 steps that can help you connect Spark to Apache Mesos.

- Configure the Spark Driver program to connect with Apache Mesos
- Put the Spark binary package in a location accessible by Mesos
- Install Spark in the same location as that of the Apache Mesos
- Configure the spark.mesos.executor.home property for pointing to the location where Spark is installed

7. What is a Parquet file and what are its advantages?

Ans:

Parquet is a columnar format that is supported by several data processing systems. With the Parquet file, Spark can perform both read and write operations.

Some of the advantages of having a Parquet file are:

- It enables you to fetch specific columns for access.
- It consumes less space
- It follows the type-specific encoding
- It supports limited I/O operations

8. What is shuffling in Spark? When does it occur?

Ans:

- Shuffling is the process of redistributing data across partitions that may lead to data movement across the executors. The shuffle operation is implemented differently in Spark compared to Hadoop.

Shuffling has 2 important compression parameters:

- `spark.shuffle.compress` – checks whether the engine would compress shuffle outputs or not
- `spark.shuffle.spill.compress` – decides whether to compress intermediate shuffle spill files or not
- It occurs while joining two tables or while performing `byKey` operations such as `GroupByKey` or `ReduceByKey`

9. What are the various functionalities supported by Spark Core?

Ans:

Spark Core is the engine for parallel and distributed processing of large data sets. The various functionalities supported by Spark Core include:

- Scheduling and monitoring jobs
- Memory management
- Fault recovery
- Task dispatching
- `eDataFrame`

10. Explain the types of operations supported by RDDs.

Ans:

RDDs support 2 types of operation:

- **Transformations:** Transformations are operations that are performed on an RDD to create a new RDD containing the results (Example: `map`, `filter`, `join`, `union`)
- **Actions:** Actions are operations that return a value after running a computation on an RDD (Example: `reduce`, `first`, `count`)

11. How to programmatically specify a schema for DataFrame?

Ans:

DataFrame can be created programmatically with three steps:

- Create an RDD of Rows from the original RDD;
- Create the schema represented by a StructType matching the structure of Rows in the RDD created in Step 1.
- Apply the schema to the RDD of Rows via createDataFrame method provided by SparkSession.

12. What is a Lineage Graph?

Ans:

- A Lineage Graph is a dependencies graph between the existing RDD and the new RDD. It means that all the dependencies between the RDD will be recorded in a graph, rather than the original data.
- The need for an RDD lineage graph happens when we want to compute new RDD or if we want to recover the lost data from the lost persisted RDD. Spark does not support data replication in memory. So, if any data is lost, it can be rebuilt using RDD lineage. It is also called an RDD operator graph or RDD dependency graph.

13. Which transformation returns a new DStream by selecting only those records of the source DStream for which the function returns true?

1. map(func)
2. transform(func)
3. filter(func)
4. count()

Ans:

- 3) filter(func).

14. Does Apache Spark provide checkpoints?

Ans:

- Yes, Apache Spark provides an API for adding and managing checkpoints. Checkpointing is the process of making streaming applications resilient to failures. It allows you to save the data and metadata into a checkpointing directory. In case of a failure, the spark can recover this data and start from wherever it has stopped.
- There are 2 types of data for which we can use checkpointing in Spark.
- **Metadata Checkpointing:** Metadata means the data about data. It refers to saving the metadata to fault-tolerant storage like HDFS. Metadata includes configurations, DStream operations, and incomplete batches.
- **Data Checkpointing:** Here, we save the RDD to reliable storage because its need arises in some of the stateful transformations. In this case, the upcoming RDD depends on the RDDs of previous batches.

15. What are the different levels of persistence in Spark?

Ans:

- **DISK_ONLY** – Stores the RDD partitions only on the disk
- **MEMORY_ONLY_SER** – Stores the RDD as serialized Java objects with one-byte array per partition
- **MEMORY_ONLY** – Stores the RDD as deserialized Java objects in the JVM. If the RDD is not able to fit in the memory available, some partitions won't be cached
- **OFF_HEAP** – Works like **MEMORY_ONLY_SER** but stores the data in off-heap memory
- **MEMORY_AND_DISK** – Stores RDD as deserialized Java objects in the JVM. In case the RDD is not able to fit in the memory, additional partitions are stored on the disk
- **MEMORY_AND_DISK_SER** – Identical to **MEMORY_ONLY_SER** with the exception of storing partitions not able to fit in the memory to the disk

16. What is the difference between map and flatMap transformation in Spark Streaming?

Ans:

map()	flatMap();
A map function returns a new DStream by passing each element of the source DStream through a function	It is similar to map function and applies to each RDD and it returns the result as new RDD
Spark Map function takes one element as input process it according to custom code (specified by the developer) and returns one element at a time	flatMap allows returning 0, 1 or more elements from function. In the flatMap operation

17. How would you compute the total count of unique words in Spark?

Ans:

1. Load the text file as RDD:

- `sc.textFile("hdfs://Hadoop/user/test_file.txt");`

2. Function that breaks each line into words:

- `def toWords(line):`
- `return line.split();`

3. Run the toWords function on each element of RDD in Spark as flatMap transformation:

- `words = line.flatMap(toWords);`

4. Convert each word into (key,value) pair:

- `def toTuple(word):`
- `return (word, 1);`
- `wordTuple = words.map(toTuple);`

5. Perform reduceByKey() action:

- `def sum(x, y):`
- `return x+y:`
- `counts = wordsTuple.reduceByKey(sum)`

6. Print:

- `counts.collect()`

18. What are the different MLlib tools available in Spark?

Ans:

- ML Algorithms: Classification, Regression, Clustering, and Collaborative filtering
- Featurization: Feature extraction, Transformation, Dimensionality reduction, and Selection
- Pipelines: Tools for constructing, evaluating, and tuning ML pipelines
- Persistence: Saving and loading algorithms, models and pipelines
- Utilities: Linear algebra, statistics, data handling

19. What is a Sparse Vector?

Ans:

A Sparse vector is a type of local vector which is represented by an index array and a value array.

- `public class SparseVector`
- `extends Object`
- `implements Vector`

1. *Example: `sparse1 = SparseVector(4, [1, 3], [3.0, 4.0])`*

2. *where:*

3. *4 is the size of the vector*

4. *[1,3] are the ordered indices of the vector*
5. *[3,4] are the value*

20. What are the functions of Spark SQL?

Ans:

- Spark SQL is Apache Spark's module for working with structured data.
 - Spark SQL loads the data from a variety of structured data sources.
 - It queries data using SQL statements, both inside a Spark program and from external tools that connect to Spark SQL through standard database connectors (JDBC/ODBC).
 - It provides a rich integration between SQL and regular Python/Java/Scala code, including the ability to join RDDs and SQL tables and expose custom functions in SQL.
-

21. What are the different types of operators provided by the Apache GraphX library?

Ans:

- **Property Operator:** Property operators modify the vertex or edge properties using a user-defined map function and produce a new graph.
- **Structural Operator:** Structure operators operate on the structure of an input graph and produce a new graph.
- **Join Operator:** Join operators add data to graphs and generate new graphs.

22. What are the analytic algorithms provided in Apache Spark GraphX?

Ans:

GraphX is Apache Spark's API for graphs and graph-parallel computation. GraphX includes a set of graph algorithms to simplify analytics tasks. The algorithms are contained in the

org.apache.spark.graphx.lib package and can be accessed directly as methods on Graph via GraphOps.

- **PageRank:** PageRank is a graph parallel computation that measures the importance of each vertex in a graph. Example: You can run PageRank to evaluate what the most important pages in Wikipedia are.
- **Connected Components:** The connected components algorithm labels each connected component of the graph with the ID of its lowest-numbered vertex. For example, in a social network, connected components can approximate clusters.
- **Triangle Counting:** A vertex is part of a triangle when it has two adjacent vertices with an edge between them. GraphX implements a triangle counting algorithm in the TriangleCount object that determines the number of triangles passing through each vertex, providing a measure of clustering.

23. Which Profilers do we use in PySpark?

Ans:

Custom profilers are PySpark supported in PySpark to allow for different Profilers to be used and for outputting to different formats than what is offered in the BasicProfiler.

We need to define or inherit the following methods, with a custom profiler:

- profile – Basically, it produces a system profile of some sort.
- stats – Well, it returns the collected stats.
- dump – Whereas, it dumps the profiles to a path.
- add – Moreover, this method helps to add a profile to the existing accumulated profile

Generally, when we create a SparkContext, we choose the profiler class.

24. Explain Basic Profiler.

Ans:

It is a default profiler, which we implement on the basis of cProfile and Accumulator.

25. Do we have a machine learning API in Python?

Ans:

As Spark provides a Machine Learning API, MLlib. Similarly, in Python as well, PySpark has this machine learning API.

26. Name algorithms supported in PySpark?

Ans:

There are several algorithms in PySpark:

- `mllib.classification`
- `mllib.clustering`
- `mllib.fpm`
- `mllib.linalg`
- `mllib.recommendation`
- `spark.mllib`
- `Mllib.regression`

27. Name parameter of SparkContext?

Ans:

The parameters of a SparkContext are:

- Master – URL of the cluster from which it connects.
- appName – Name of our job.
- sparkHome – Spark installation directory.
- pyFiles – It is the .zip or .py files, in order to send to the cluster and also to add to the PYTHONPATH.
- Environment – Worker nodes environment variables.
- Serializer – RDD serializer.
- Conf – to set all the Spark properties, an object of L{SparkConf}.
- JSC – It is the JavaSparkContext instance.

28. Which of the parameters of SparkContext we mostly use?

Ans:

Master and app name.

29. What Makes Apache Spark Good At Low-latency Workloads Like Graph Processing And Machine Learning?

Ans:

Apache Spark stores data in-memory for faster model building and training. Machine learning algorithms require multiple iterations to generate a resulting optimal model and similarly graph algorithms traverse all the nodes and edges. These low latency workloads that need multiple iterations can lead to increased performance. Less disk access and controlled network traffic make a huge difference when there is lots of data to be processed.

30. Is It Necessary To Start Hadoop To Run Any Apache Spark Application ?

Ans:

Starting hadoop is not mandatory to run any spark application. As there is no separate storage in Apache Spark, it uses Hadoop HDFS but it is not mandatory. The data can be stored in the local file system, can be loaded from the local file system and processed.

31. What Is The Default Level Of Parallelism In Apache Spark?

Ans:

If the user does not explicitly specify then the number of partitions are considered as default level of parallelism in Apache Spark.

32. Explain About The Common Workflow Of A Spark Program

Ans:

- The foremost step in a Spark program involves creating input RDD's from external data.
- Use various RDD transformations like filter() to create new transformed RDD's based on the business logic.
- persist() any intermediate RDD's which might have to be reused in future.
- Launch various RDD actions() like first(), count() to begin parallel computation , which will then be optimized and executed by Spark.

33. Name A Few Commonly Used Spark Ecosystems.

Ans:

- Spark SQL (Shark)
- Spark Streaming
- GraphX
- MLlib
- SparkR

34. What is Spark Streaming?

Ans:

At whatever point there is information streaming constantly and you need to process the information as right on time as could reasonably be expected, all things considered you can exploit Spark Streaming.

35. Can We Do Real-time Processing Using Spark Sql?

Ans:

Not directly but we can register an existing RDD as a SQL table and trigger SQL queries on top of that.

36. What Is Spark Sql?

Ans:

SQL Spark, better known as Shark is a novel module introduced in Spark to work with structured data and perform structured data processing. Through this module, Spark executes relational SQL queries on the data. The core of the component supports an altogether different RDD called SchemaRDD, composed of rows objects and schema objects defining data type of each column in the row. It is similar to a table in a relational database.

37. What Is A Parquet File?

Ans:

Parquet is a columnar format file supported by many other data processing systems. Spark SQL performs both read and write operations with Parquet files and considers it to be one of the best big data analytics formats so far.

38. List The Functions Of Spark Sql.

Ans:

Spark SQL is capable of:

- Loading data from a variety of structured sources
- Querying data using SQL statements, both inside a Spark program and from external tools that connect to Spark SQL through standard database connectors (JDBC/ODBC). For instance, using business intelligence tools like Tableau
- Providing rich integration between SQL and regular Python/Java/Scala code, including the ability to join RDDs and SQL tables, expose custom functions in SQL, and more

39. What Is Spark?

Ans:

Spark is a parallel data processing framework. It allows developers to develop fast, unified big data applications that combine batch, streaming and interactive analytics.

40. What Is Hive On Spark?

Ans:

- Hive is a component of Hortonworks' Data Platform (HDP). Hive provides an SQL-like interface to data stored in the HDP. Spark users will automatically get the complete set of Hive's rich features, including any new features that Hive might introduce in the future.
- The main task around implementing the Spark execution engine for Hive lies in query planning, where Hive operator plans from the semantic analyzer which is translated to a task plan that Spark can execute. It also includes query execution, where the generated Spark plan gets actually executed in the Spark cluster.

41. What Is A "Parquet" In Spark?

Ans:

“Parquet” is a columnar format file supported by many data processing systems. Spark SQL performs both read and write operations with the “Parquet” file.

42. What Are Benefits Of Spark Over Mapreduce?

Ans:

Due to the availability of in-memory processing, Spark implements the processing around 10-100x faster than Hadoop MapReduce. MapReduce makes use of persistence storage for any of the data processing tasks.

- Unlike Hadoop, Spark provides in-built libraries to perform multiple tasks from the same core like batch processing, Streaming, Machine learning, Interactive SQL queries. However, Hadoop only supports batch processing.
- Hadoop is highly disk-dependent whereas Spark promotes caching and in-memory data storage
- Spark is capable of performing computations multiple times on the same dataset. This is called iterative computation while there is no iterative computing implemented by Hadoop.

43. How Spark Sql Is Different From Hql And Sql?

Ans:

SparkSQL is a special component on the spark Core engine that supports SQL and Hive Query Language without changing any syntax. It's possible to join SQL table and HQL table.

44. What do you understand about SchemaRDD in Apache Spark RDD?

Ans:

- *SchemaRDD* is an RDD that consists of row objects (wrappers around the basic string or integer arrays) with schema information about the type of data in each column.
- SchemaRDD was designed as an attempt to make life easier for developers in their daily routines of code debugging and unit testing on SparkSQL core module. The idea can boil down to describing the data structures inside RDD using a formal description similar to the relational database schema. On top of all basic functions provided by common RDD APIs, SchemaRDD also provides some straightforward relational query interface functions that are realized through SparkSQL.
- Now, it is officially renamed to *DataFrame API* on Spark's latest trunk.

45. How is Spark SQL different from HQL and SQL?

Ans:

Spark SQL is a special component on the Spark Core engine that supports SQL and Hive Query Language without changing any syntax. It is possible to join SQL table and HQL table to Spark SQL.

46. What is a DStream in Apache Spark?

Ans:

- *Discretized Stream* (DStream) is the basic abstraction provided by Spark Streaming. It is a continuous stream of data. It is received from a data source or from a processed data stream generated by transforming the input stream. Internally, a DStream is represented by a continuous

series of RDDs and each RDD contains data from a certain interval. Any operation applied on a DStream translates to operations on the underlying RDDs.

DStreams can be created from various sources like Apache Kafka, HDFS, and Apache Flume.

DStreams have two operations:

- Transformations that produce a new DStream.
- Output operations that write data to an external system.

There are many DStream transformations possible in Spark Streaming. Let us look at `filter(func)`. `filter(func)` returns a new DStream by selecting only the records of the source DStream on which `func` returns true.

47. When running Spark applications, is it necessary to install Spark on all the nodes of the YARN cluster?

Ans:

Spark need not be installed when running a job under YARN or Mesos because Spark can execute on top of YARN or Mesos clusters without affecting any change to the cluster.

48. What are the various data sources available in Spark SQL?

Ans:

Parquet file, JSON datasets and Hive tables are the data sources available in Spark SQL.

49. What are the various levels of persistence in Apache Spark?

Ans:

Apache Spark automatically persists the intermediary data from various shuffle operations, however, it is often suggested that users call `persist()` method on the RDD in case they plan to reuse it. Spark has various persistence levels to store the RDDs on disk or in memory or as a combination of both with different replication levels.

The various storage/persistence levels in Spark are:

- **MEMORY_ONLY:** Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
- **MEMORY_AND_DISK:** Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
- **MEMORY_ONLY_SER:** Store RDD as *serialized* Java objects (one byte array per partition).
- **MEMORY_AND_DISK_SER:** Similar to **MEMORY_ONLY_SER**, but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
- **DISK_ONLY:** Store the RDD partitions only on disk.
- **OFF_HEAP:** Similar to **MEMORY_ONLY_SER**, but store the data in off-heap memory.

50. How Spark uses Akka?

Ans:

Spark uses Akka basically for scheduling. All the workers request for a task to master after registering. The master just assigns the task. Here Spark uses Akka for messaging between the workers and masters.

51. What do you understand by Lazy Evaluation?

Spark is intellectual in the manner in which it operates on data. When you tell Spark to operate on a given dataset, it heeds the instructions and makes a note of it, so that it does not forget – but it does nothing, unless asked for the final result. When a transformation like `map()` is called on an RDD, the operation is not performed immediately. Transformations in Spark are not evaluated till you perform an action. This helps optimize the overall data processing workflow.

52. How is AI executed in Spark?

Ans:

MLlib is an adaptable AI library given by Spark. It goes for making AI simple and adaptable with normal learning calculations and use cases like bunching, relapse separating, dimensional decrease, and alike.

53. What is Spark Executor?

Ans:

At the point when SparkContext associates with a group chief, it obtains an Executor on hubs in the bunch. Representatives are Spark forms that run controls and store the information on the laborer hub. The last assignments by SparkContext are moved to agents for their execution.

54. Name kinds of Cluster Managers in Spark.

Ans:

The Spark system underpins three noteworthy sorts of Cluster Managers:

- **Standalone:** An essential administrator to set up a group.
- **Apache Mesos:** Generalized/regularly utilized group administrator, additionally runs Hadoop MapReduce and different applications.
- **YARN:** Responsible for asset the board in Hadoop.

55. Show some utilization situations where Spark beats Hadoop in preparing.

Ans:

- **Sensor Data Processing:** Apache Spark's "In-memory" figuring works best here, as information is recovered and joined from various sources.
- **Real Time Processing:** Spark is favored over Hadoop for constant questioning of information. for example Securities exchange Analysis, Banking, Healthcare, Telecommunications, and so on.
- **Stream Processing:** For preparing logs and identifying cheats in live streams for cautions, Apache Spark is the best arrangement.
- **Big Data Processing:** Spark runs upto multiple times quicker than Hadoop with regards to preparing medium and enormous estimated datasets.

56. By what method can Spark be associated with Apache Mesos?

Ans:

To associate Spark with Mesos:

- Configure the sparkle driver program to associate with Mesos.

- Spark paired bundle ought to be in an area open by Mesos.
- Install Apache Spark in a similar area as that of Apache Mesos and design the property 'spark.mesos.executor.home' to point to the area where it is introduced.

57. How is Spark SQL not the same as HQL and SQL?

Ans:

Flash SQL is a unique segment on the Spark Core motor that supports SQL and Hive Query Language without changing any sentence structure. It is conceivable to join SQL table and HQL table to Spark SQL.

58. What is ancestry in Spark? How adaptation to internal failure is accomplished in Spark utilizing Lineage Graph?

Ans:

- At whatever point a progression of changes are performed on a RDD, they are not assessed promptly, however languidly.
- At the point when another RDD has been made from a current RDD every one of the conditions between the RDDs will be signed in a diagram.
- This chart is known as the ancestry diagram.
- Consider the underneath situation

Ancestry chart of every one of these activities resembles:

- First RDD
- Second RDD (applying map)
- Third RDD (applying channel)
- Fourth RDD (applying check)

This heredity diagram will be helpful on the off chance that if any of the segments of information is lost.

- Need to set spark.logLineage to be consistent with empowering the Rdd.toDebugString() gets empowered to print the chart logs.

59. What is the contrast between RDD , DataFrame and DataSets?

Ans:

RDD :

- It is the structure square of Spark. All Dataframes or Dataset is inside RDDs.
- It is lethargically assessed permanent gathering objects
- RDDS can be effectively reserved if a similar arrangement of information should be recomputed.

DataFrame :

- Gives the construction see (lines and segments). It tends to be thought of as a table in a database.
- Like RDD even the dataframe is sluggishly assessed.
- It offers colossal execution due to a.) Custom Memory Management – Data is put away in off load memory in twofold arrangement .No refuse accumulation because of this.
- **Optimized Execution Plan** – Query plans are made utilizing Catalyst analyzer.
- **DataFrame Limitations** : Compile Time wellbeing , i.e no control of information is conceivable when the structure isn't known.
- **DataSet** : Expansion of DataFrame
- **DataSet Features** – Provides best encoding component and not at all like information edges supports arrange time security.

60. What is DStream?

Ans:

- Discretized Stream (DStream)
- Apache Spark Discretized Stream is a gathering of RDDs in grouping .
- Essentially, it speaks to a flood of information or gathering of Rdds separated into little clusters. In addition, DStreams are based on Spark RDDs, Spark's center information reflection. It likewise enables Streaming to flawlessly coordinate with some other Apache Spark segments. For example, Spark MLlib and Spark SQL.

61. What is the connection between Job, Task, Stage ?

Ans:

- **Errand**

An errand is a unit of work that is sent to the agent. Each stage has some assignment, one undertaking for every segment. The Same assignment is done over various segments of RDD.

- **Occupation**

The activity is a parallel calculation consisting of numerous undertakings that get produced in light of activities in Apache Spark.

- **Stage**

Each activity gets isolated into littler arrangements of assignments considered stages that rely upon one another. Stages are named computational limits. All calculation is impossible in a single stage. It is accomplished over numerous stages.

62. Clarify quickly about the parts of Spark Architecture?

Ans:

Flash Driver: The Spark driver is the procedure running the sparkle setting . This driver is in charge of changing over the application to a guided diagram of individual strides to execute on the bunch. There is one driver for each application.

63. How might you limit information moves when working with Spark?

Ans:

- The different manners by which information moves can be limited when working with Apache Spark are:
- Communicate and Accumulator factors

64. When running Spark applications, is it important to introduce Spark on every one of the hubs of YARN group?

Ans:

Flash need not be introduced when running a vocation under YARN or Mesos in light of the fact that Spark can execute over YARN or Mesos bunches without influencing any change to the group.

65. Which one will you decide for an undertaking – Hadoop MapReduce or Apache Spark?

Ans:

The response to this inquiry relies upon the given undertaking situation – as it is realized that Spark utilizes memory rather than system and plate I/O. In any case, Spark utilizes enormous measure of RAM and requires devoted machines to create viable outcomes. So the choice to utilize Hadoop or Spark changes powerfully with the necessities of the venture and spending plan of the association.

66. What is the distinction among continue() and store()

Ans:

endure () enables the client to determine the capacity level while reserve () utilizes the default stockpiling level.

67. What are the different dimensions of constancy in Apache Spark?

Ans:

Apache Spark naturally endures the mediator information from different mix tasks, anyway it is regularly proposed that clients call persevere () technique on the RDD on the off chance that they intend to reuse it. Sparkle has different tirelessness levels to store the RDDs on circle or in memory or as a mix of both with various replication levels.

68. What are the disservices of utilizing Apache Spark over Hadoop MapReduce?

Ans:

Apache Spark's in-memory ability now and again comes a noteworthy barrier for cost effective preparing of huge information. Likewise, Spark has its own record the board framework and consequently should be incorporated with other cloud based information stages or apache hadoop.

69. What is the upside of Spark apathetic assessment?

Ans:

Apache Spark utilizes sluggish assessment all together the advantages:

- Applying Transformations tasks on RDD or "stacking information into RDD" isn't executed quickly until it sees an activity. Changes on RDDs and putting away information in RDD are languidly assessed. Assets will be used in a superior manner if Spark utilizes sluggish assessment.

- Lazy assessment advances the plate and memory utilization in Spark.
- The activities are activated just when the information is required. It diminishes overhead.

70. What are the advantages of Spark over MapReduce?

Ans:

- Because of the accessibility of in-memory handling, Spark executes the preparing around 10 to multiple times quicker than Hadoop MapReduce while MapReduce utilizes diligence stockpiling for any of the information handling errands.
- Dissimilar to Hadoop, Spark gives inbuilt libraries to play out numerous errands from a similar center like cluster preparing, Steaming, Machine learning, Interactive SQL inquiries. Be that as it may, Hadoop just backings cluster handling.
- Hadoop is very plate subordinate while Spark advances reserving and in-memory information stockpiling.

71. How DAG functions in Spark?

Ans:

- At the point when an Action is approached Spark RDD at an abnormal state, Spark presents the heredity chart to the DAG Scheduler.
- Activities are separated into phases of the errand in the DAG Scheduler. A phase contains errands dependent on the parcel of the info information. The DAG scheduler pipelines administrators together. It dispatches tasks through the group chief. The conditions of stages are obscure to the errand scheduler. The Workers execute the undertaking on the slave.

72. What is the hugeness of the Sliding Window task?

Ans:

Sliding Window controls transmission of information bundles between different PC systems. Sparkle Streaming library gives windowed calculations where the changes on RDDs are connected over a sliding window of information. At whatever point the window slides, the RDDs that fall inside the specific window are consolidated and worked upon to create new RDDs of the windowed DStream.

73. What are communicated and Accumulators?

Ans:

- **Communicate variable:**

On the off chance that we have an enormous dataset, rather than moving a duplicate of informational collection for each assignment, we can utilize a communication variable which can be replicated to every hub at one time and share similar information for each errand in that hub. Communicate variable assistance to give a huge informational collection to every hub.

- **Collector:**

Flash capacities utilized factors characterized in the driver program and nearby replicated factors will be produced. Aggregators are shared factors which help to refresh factors in parallel during execution and offer the outcomes from specialists to the driver.

74. What are activities ?

Ans:

An activity helps in bringing back the information from RDD to the nearby machine. An activity's execution is the aftereffect of all recently made changes. `persist()` is an activity that executes the capacity passed over and over until one esteem assuming left. `take()` move makes every one of the qualities from RDD to nearby hub.

75. What is YARN?

Ans:

Like Hadoop, YARN is one of the key highlights in Spark, giving a focal and asset the executives stage to convey adaptable activities over the bunch. YARN is a conveyed holder chief, as Mesos for instance, while Spark is an information preparing instrument. Sparkle can keep running on YARN, a similar way Hadoop Map Reduce can keep running on YARN. Running Spark on YARN requires a double dispersion of Spark as based on YARN support.

76. Clarify the key highlights of Apache Spark.

Ans:

Coming up next are the key highlights of Apache Spark:

- Polyglot
- Speed
- Multiple Format Support
- Lazy Evaluation
- Hadoop Integration
- Machine Learning

77. How is Streaming executed in Spark? Clarify with precedents.

Ans:

Sparkle Streaming is utilized for handling constant gushing information. Along these lines it is a helpful expansion into the Spark API. It empowers high-throughput and shortcoming tolerant stream handling of live information streams. The crucial stream unit is DStream which is fundamentally a progression of RDDs (Resilient Distributed Datasets) to process the constant information. The information from various sources like Flume, HDFS is spilled lastly to document frameworks, live

dashboards and databases. It is like a bunch preparing as the information is partitioned into streams like clusters.

78. What are the enhancements that engineers can make while working with flash?

Ans:

- Flash is memory serious, whatever you do it does in memory.
- Initially, you can alter to what extent flash will hold up before it times out on every one of the periods of information region information neighborhood process nearby hub nearby rack neighborhood Any.
- Channel out information as ahead of schedule as could be allowed. For reserving, pick carefully from different capacity levels.
- Tune the quantity of parcels in sparkle.

79. List some use cases where Spark outperforms Hadoop in processing.

Ans:

- Sensor Data Processing: Apache Spark's "In-memory" computing works best here, as data is retrieved and combined from different sources.
- Real Time Processing: Spark is preferred over Hadoop for real-time querying of data. e.g. Stock Market Analysis, Banking, Healthcare, Telecommunications, etc.
- Stream Processing: For processing logs and detecting frauds in live streams for alerts, Apache Spark is the best solution.
- Big Data Processing: Spark runs upto 100 times faster than Hadoop when it comes to processing medium and large-sized datasets.

80. What is a Data Frame?

Ans:

An information casing resembles a table, it got some named sections which are composed into segments. You can make an information outline from a document or from tables in hive, outside databases SQL or NoSQL or existing RDD's. It is practically equivalent to a table.

81. How might you associate Hive to Spark SQL?

Ans:

- The principal significant thing is that you need to place the hive-site.xml record in the conf index of Spark.
- At that point with the assistance of Spark session object we can develop an information outline as,

82. What is GraphX?

Ans:

- Ordinarily you need to process the information as charts, since you need to do some examination on it. It endeavors to perform Graph calculation in Spark in which information is available in documents or in RDD's.
- GraphX is based on the highest point of Spark center, so it has got every one of the abilities of Apache Spark like adaptation to internal failure, scaling and there are numerous inbuilt chart calculations too. GraphX binds together ETL, exploratory investigation and iterative diagram calculation inside a solitary framework.
- You can see indistinguishable information from the two charts and accumulations, change and unite diagrams with RDD effectively and compose custom iterative calculations utilizing the pregel API.
- GraphX contends on execution with the quickest diagram frameworks while holding Spark's adaptability, adaptation to internal failure and convenience.

Apache Spark Life Cycle Explained with Code

Let's walk through a simple Apache Spark application to understand its life cycle. We'll use PySpark (the Python API for Spark) in this example.

```
from pyspark import SparkConf, SparkContext

# Step 1: SparkContext Creation
conf = SparkConf().setAppName("WordCountApp").setMaster("local")
sc = SparkContext(conf=conf)

# Step 2: Loading Data
input_file = "sample_text.txt"
text_rdd = sc.textFile(input_file)

# Step 3: Transformation - Splitting Lines into Words
words_rdd = text_rdd.flatMap(lambda line: line.split(" "))

# Step 4: Transformation - Mapping Words to (Word, 1) Pairs
word_pairs_rdd = words_rdd.map(lambda word: (word, 1))

# Step 5: Action - Reducing by Key to Count Words
word_count_rdd = word_pairs_rdd.reduceByKey(lambda a, b: a + b)

# Step 6: Action - Collecting Results
word_counts = word_count_rdd.collect()

# Step 7: Printing Results
for word, count in word_counts:
    print(f"{word}: {count}")

# Step 8: Stopping the SparkContext
sc.stop()
```

Life Cycle of an Apache Spark Application

1. SparkContext Creation:

- **Code:** `conf = SparkConf().setAppName("WordCountApp").setMaster("local")` and `sc = SparkContext(conf=conf)`

- **Explanation:** The first step in any Spark application is to create a `SparkContext`, which serves as the entry point for interacting with the Spark cluster. It tells Spark how to access a cluster and initializes the necessary resources. Here, we set the application name to "WordCountApp" and run it locally (`local`).

2. Loading Data:

- **Code:** `text_rdd = sc.textFile(input_file)`
- **Explanation:** Once the `SparkContext` is created, the application can load data. In this step, we load a text file into an RDD (Resilient Distributed Dataset) using `textFile()`. RDDs are the fundamental data structure in Spark, representing a distributed collection of data.

3. Transformation - Splitting Lines into Words:

- **Code:** `words_rdd = text_rdd.flatMap(lambda line: line.split(" "))`
- **Explanation:** Transformations in Spark are operations on RDDs that return a new RDD. Here, we use `flatMap()` to split each line of the text file into words. `flatMap()` is a transformation that applies the given function to all elements of the RDD and returns a new RDD.

4. Transformation - Mapping Words to (Word, 1) Pairs:

- **Code:** `word_pairs_rdd = words_rdd.map(lambda word: (word, 1))`
- **Explanation:** We then map each word to a key-value pair where the key is the word, and the value is 1. The `map()` transformation returns a new RDD with these key-value pairs.

5. Action - Reducing by Key to Count Words:

- **Code:** `word_count_rdd = word_pairs_rdd.reduceByKey(lambda a, b: a + b)`
- **Explanation:** Actions are operations that trigger the actual computation of RDDs. Here, `reduceByKey()` is an action that aggregates the values of each key (word) by summing the counts. This results in an RDD where each word is associated with its count.

6. Action - Collecting Results:

- **Code:** `word_counts = word_count_rdd.collect()`
- **Explanation:** The `collect()` action brings all the elements of the RDD to the driver program as a list. This is typically used when you want to inspect the final results in the driver.

7. Printing Results:

- **Code:** `for word, count in word_counts: print(f"{word}: {count}")`
- **Explanation:** Finally, we loop through the results and print the word counts. This step is part of the application logic rather than Spark's core operations.

8. Stopping the SparkContext:

- **Code:** `sc.stop()`
- **Explanation:** The last step in the Spark life cycle is stopping the `SparkContext`. This releases the resources and shuts down the application.

Summary of Spark Life Cycle Steps:

1. **SparkContext Creation:** Initializes the connection to the cluster.
2. **Job Submission:** Occurs when actions like `reduceByKey()` or `collect()` are invoked.
3. **Stage Creation:** Spark divides the job into stages, which are a series of tasks based on the RDD lineage and shuffle boundaries.
4. **Task Execution:** Each stage consists of tasks that are executed in parallel across the cluster nodes.
5. **Result Collection:** The results of the computation are collected back to the driver.
6. **Stopping SparkContext:** The application terminates, and resources are released.

This life cycle ensures that Spark applications are efficiently managed from start to finish, allowing for the scalable and fast processing of large datasets.

Here are some valuable resources for learning more about Apache Spark and its application:

1. Official Documentation:

- [Apache Spark Documentation](#): The official Spark documentation provides comprehensive details on Spark's features, APIs, and how to get started.

2. Books:

- **"Learning Spark: Lightning-Fast Data Analytics" by Jules Damji, et al.:** This book covers the fundamentals of Apache Spark, including the Spark application life cycle, and offers hands-on examples.
- **"Spark: The Definitive Guide" by Bill Chambers and Matei Zaharia:** Written by one of the creators of Spark, this book is an in-depth resource for mastering Spark.

3. Online Courses:

- [Coursera - Big Data Analysis with Spark](#): This course provides practical experience with Spark, including application development and optimization techniques.
- **edX - Introduction to Apache Spark**: An introductory course that helps learners understand the basics of Spark and its ecosystem.

4. Tutorials and Blogs:

- **Databricks Spark Tutorial**: A great place to start with hands-on Spark tutorials.
- **Towards Data Science - Apache Spark Series**: This blog series covers various aspects of Spark, including optimization tips and use cases.

5. GitHub Repositories:

- [Apache Spark GitHub Repository](#): Explore the source code of Apache Spark, which can be useful for understanding its internals.
- [Awesome Spark](#): A curated list of awesome Spark libraries, resources, and tutorials.

6. Community Forums and Discussions:

- [Stack Overflow - Apache Spark](#): A community-driven platform where you can ask questions and find answers related to Spark.
- [Apache Spark User Mailing List](#): Join the community mailing list to stay updated and engage with other Spark users.

These resources should help you dive deeper into Apache Spark and its life cycle, as well as improve your ability to develop and optimize Spark applications.

All Spark Optimizations with code <#>

1. Partitioning <#>

Explanation <#>

Partitioning refers to dividing the data into smaller, manageable chunks (partitions) across the cluster's nodes. Proper partitioning ensures parallel processing and avoids data skew, leading to balanced workloads and improved performance.

Code Example <#>

```
# Repartitioning DataFrame to 10 partitions based on a column
df_repartitioned = df.repartition(10, "column_name")
```

2. Caching and Persistence <#>

Explanation <#>

Caching and persistence are used to store intermediate results in memory, reducing the need for recomputation. This is particularly useful when the same DataFrame is accessed multiple times in a Spark job.

Code Example <#>

```
# Caching DataFrame in memory
df.cache()
df.show()

# Persisting DataFrame with a specific storage level (Memory and Disk)
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)
df.show()
```

3. Broadcast Variables <#>

Explanation <#>

Broadcast variables allow the distribution of a read-only variable to all nodes in the cluster, which can be more efficient than shipping the variable with every task. This is particularly useful for small lookup tables.

Code Example <#>

```
# Broadcasting a variable
broadcastVar = sc.broadcast([1, 2, 3])
```

4. Avoiding Shuffles <#>

Explanation <#>

Shuffles are expensive operations that involve moving data across the cluster. Minimizing shuffles by using map-side combine or careful partitioning can significantly improve performance.

Code Example <#>

```
# Using map-side combine to reduce shuffle
rdd = rdd.map(lambda x: (x, 1)).reduceByKey(lambda x, y: x + y)
```

5. Columnar Format <#>

Explanation <#>

Using columnar storage formats like Parquet or ORC can improve read performance by allowing Spark to read only the necessary columns. These formats also support efficient compression and encoding schemes.

Code Example <#>

```
# Saving DataFrame as Parquet

df.write.parquet("path/to/parquet/file")
```

6. Predicate Pushdown <#>

Explanation <#>

Predicate pushdown allows Spark to filter data at the data source level before loading it into memory, reducing the amount of data transferred and improving performance.

Code Example <#>

```
# Reading data with predicate pushdown
df = spark.read.parquet("path/to/parquet/file").filter("column_name > 100")
```

7. Vectorized UDFs (Pandas UDFs) <#>

Explanation <#>

Vectorized UDFs, also known as Pandas UDFs, utilize Apache Arrow to process batches of rows, improving performance compared to row-by-row processing in standard UDFs.

Code Example <#>

```
from pyspark.sql.functions
import pandas_udf, PandasUDFType

@pandas_udf("double", PandasUDFType.SCALAR)
def vectorized_udf(x):
    return x + 1

df.withColumn("new_column", vectorized_udf(df["existing_column"])).show()
```

8. Coalesce <#>

Explanation <#>

Coalesce reduces the number of partitions in a DataFrame, which is more efficient than repartitioning when decreasing the number of partitions.

Code Example <#>

```
# Coalescing DataFrame to 1 partition
df_coalesced = df.coalesce(1)
```

9. Avoid Using Explode <#>

Explanation <#>

Explode is an expensive operation that flattens arrays into multiple rows. Using it should be minimized, or optimized by reducing the size of the DataFrame before exploding.

Code Example <#>

```
# Using explode function
from pyspark.sql.functions import explode
df_exploded = df.withColumn("exploded_column", explode(df["array_column"]))
```

10. Tungsten Execution Engine <#>

Explanation <#>

Tungsten is Spark's in-memory computation engine that optimizes the execution plans for DataFrames and Datasets, utilizing memory and CPU more efficiently.

Code Example <#>

Tungsten is enabled by default in Spark, so no specific code is needed. However, using DataFrames and Datasets ensures you leverage Tungsten's optimizations.

11. Using DataFrames/Datasets API <#>

Explanation <#>

The DataFrames/Datasets API provides higher-level abstractions and optimizations compared to RDDs, including Catalyst Optimizer for query planning and execution.

Code Example <#>

```
# Using DataFrames API
df = spark.read.csv("path/to/csv/file")
df = df.groupBy("column_name").agg({"value_column": "sum"})
```


12. Join Optimization <#>

Explanation <#>

Broadcast joins are more efficient than shuffle joins when one of the DataFrames is small, as the small DataFrame is broadcasted to all nodes, avoiding shuffles.

Code Example <#>

```
# Broadcast join
from pyspark.sql.functions import broadcast df =
df1.join(broadcast(df2), df1["key"] == df2["key"])
```

13. Resource Allocation <#>

Explanation <#>

Properly allocating resources such as executor memory and cores ensures optimal performance by matching the resource requirements of your Spark jobs.

Code Example <#>

Resource allocation is typically done through Spark configurations when submitting jobs:

```
spark-submit --executor-memory 4g --executor-cores 2 your_script.py
```

14. Skew Optimization <#>

Explanation <#>

Handling skewed data can improve performance. Techniques like salting (adding a random key) can help distribute skewed data more evenly across partitions.

Code Example <#>

```
# Handling skewed data by salting from
pyspark.sql.functions import rand

df_salted = df.withColumn("salt", (rand() * 10).cast("int"))
df_salted_repartitioned = df_salted.repartition("salt")
```

15. Speculative Execution <#>

Explanation <#>

Speculative execution re-runs slow tasks in parallel and uses the result of the first completed task, helping to mitigate the impact of straggler tasks.

Code Example <#>

```
# Enabling speculative execution

spark.conf.set("spark.speculation", "true")
```

[Ref: codeinspark.com](https://codeinspark.com)

16. Adaptive Query Execution (AQE) <#>

Explanation <#>

AQE optimizes query execution plans dynamically based on runtime statistics, such as the actual size of data processed, leading to more efficient query execution.

Code Example <#>

```
# Enabling AQE
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

17. Dynamic Partition Pruning <#>

Explanation <#>

Dynamic Partition Pruning improves the performance of join queries by dynamically pruning partitions at runtime, reducing the amount of data read.

Code Example <#>

```
# Enabling dynamic partition pruning
spark.conf.set("spark.sql.dynamicPartitionPruning.enabled", "true")
```

18. Reduce Task Serialization Overhead <#>

Explanation <#>

Using Kryo serialization can reduce the overhead associated with task serialization, improving performance compared to the default Java serialization.

Code Example <#>

```
# Enabling Kryo serialization
spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

19. Reduce Shuffle Partitions <#>

Explanation <#>

By default, Spark has a high number of shuffle partitions (200). Reducing this number can improve performance, especially for small datasets.

Code Example <#>

```
# Reducing shuffle partitions
spark.conf.set("spark.sql.shuffle.partitions", "50")
```

20. Using Data Locality <#>

Explanation <#>

Ensuring that data processing happens as close to the data as possible reduces network I/O, leading to faster data processing.

Code Example <#>

Data locality is handled by Spark's execution engine, but users can influence it by configuring their cluster properly and using locality preferences in their code.

21. Leveraging Built-in Functions <#>

Explanation <#>

Built-in functions are optimized for performance and should be preferred over custom UDFs, which can introduce significant overhead.

Code Example <#>

```
# Using built-in functions
from pyspark.sql.functions import col, expr
df.select(col("column_name").alias("new_column_name")).show()
```

This document provides detailed explanations and code examples for various Spark optimization techniques. Applying these optimizations can significantly improve the performance and efficiency of your Spark jobs. If you need more specific examples or have further questions, feel free to ask!